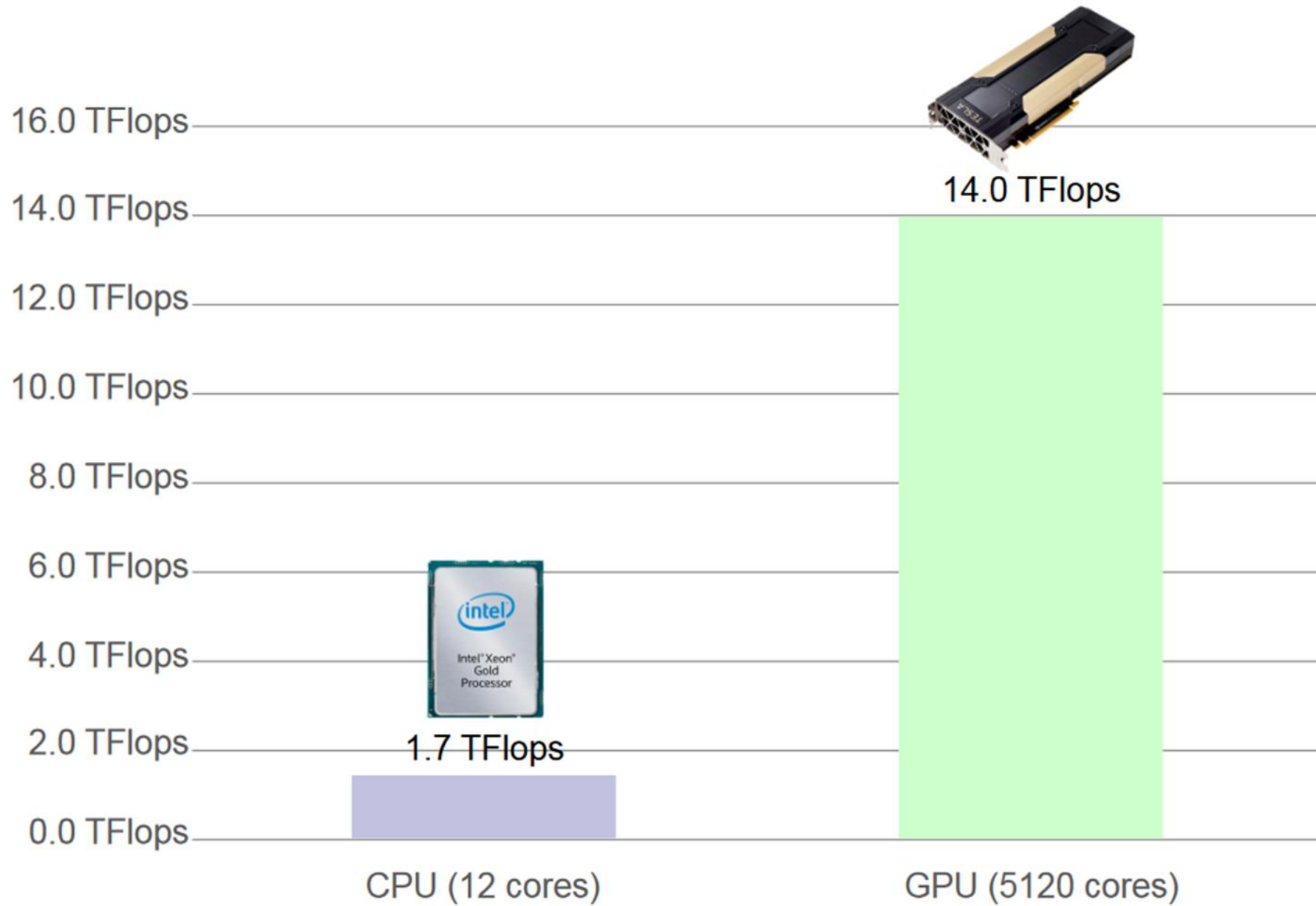
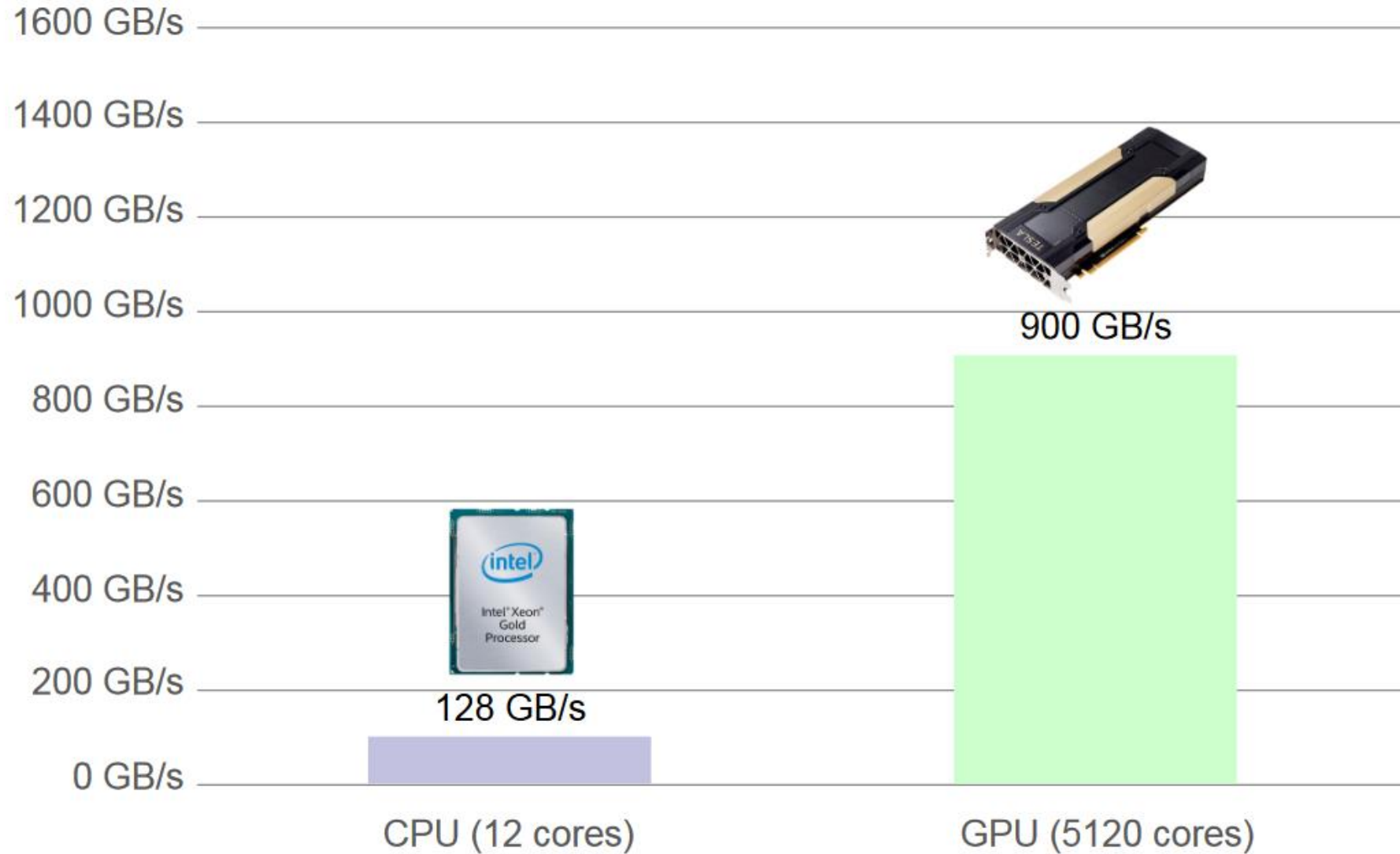


Week 9 – Numba and GPU computing

02613 Python and High-Performance Computing





Today

GPU Computing with CUDA

- How GPUs work
- How GPUs are different from CPUs
- How to code for GPUs

Gateway: Numba





What is that?

Numba is an open source **JIT compiler** that translates a subset of Python and NumPy code into fast machine code.

What is a JIT compiler?

JIT = Just In Time

Normally (for compiled code like C++):



JIT:



What is a JIT compiler?

JIT = Just In Time

Faster because:

- Machine code much faster than Python
- Can cache compiled code for next call

Normally (for compiled code like C++):



JIT:



JIT ecosystem (with GPU focus)



Triton



Numba CPU example

```
import numpy as np

def dot(x, y):
    d = 0.0
    for i in range(x.size):
        d += x[i] * y[i]
    return d
```

Time for random 100,000-d vectors
dot 21.5 ms

Numba CPU example

Convert *everything* to machine code

```
import numpy as np
from numba import jit

@jit(nopython=True)
def dot_jit(x, y):
    d = 0.0
    for i in range(x.size):
        d += x[i] * y[i]
    return d
```

Decorator

Time for random 100,000-d vectors

dot	21.5 ms
dot_jit	57.9 ms



Numba CPU example

Convert *everything* to machine code

```
import numpy as np  
from numba import jit  
  
@jit(nopython=True)
```

Time for random 100,000-d vectors	
dot	21.5 ms
dot_jit	57.9 ms

First run will be slower!
That's when Numba compiles the function



Numba CPU example

Convert *everything* to machine code

```
import numpy as np
from numba import jit

@jit(nopython=True)
def dot_jit(x, y):
    d = 0.0
    for i in range(x.size):
        d += x[i] * y[i]
    return d
```

Decorator

Time for random 100,000-d vectors

dot 21.5 ms

dot_jit 0.1 ms

Speed-up: ~200x

Numba CPU example

Convert *everything* to machine code

```
import numpy as np
from numba import jit

@jit(nopython=True)
def dot_jit(x, y):
    d = 0.0
    for i in range(x.size):
        d += x[i] * y[i]
    return d
```

Decorator

Time for random 100,000-d vectors

dot	21.5 ms
dot_jit	0.1 ms
Speed-up: ~200x	
np.dot	0.06 ms

Numba CPU example

Convert *everything* to machine code

```
import numpy as np
from numba import jit

@jit(nopython=True)
def dot_jit(x, y):
    d = 0.0
    for i in range(x.size):
        d += x[i] * y[i]
    return d
```

Decorator

Time for random 10,00,000-d vectors

dot	2000.0 ms	
dot_jit	59.6 ms	(first call)
dot_jit	16.8 ms	(subsequent)
np.dot	6.3 ms	

Numba CPU example

Convert *everything* to machine code

```
import numpy as np  
from numba import jit
```

```
@jit(nopython=True)
```

```
def dot_jit(x, y):
```

Time for random 10,00,000-d vectors

dot 2000.0 ms

dot_jit 59.6 ms (first call)

dot_jit 16.8 ms (subsequent)

Decorator

**Compilation overhead is ~constant.
Just beware when you time!**

Numba CPU example

Convert *everything* to machine code

```
import numpy as np
from numba import jit

@jit(nopython=True)
```

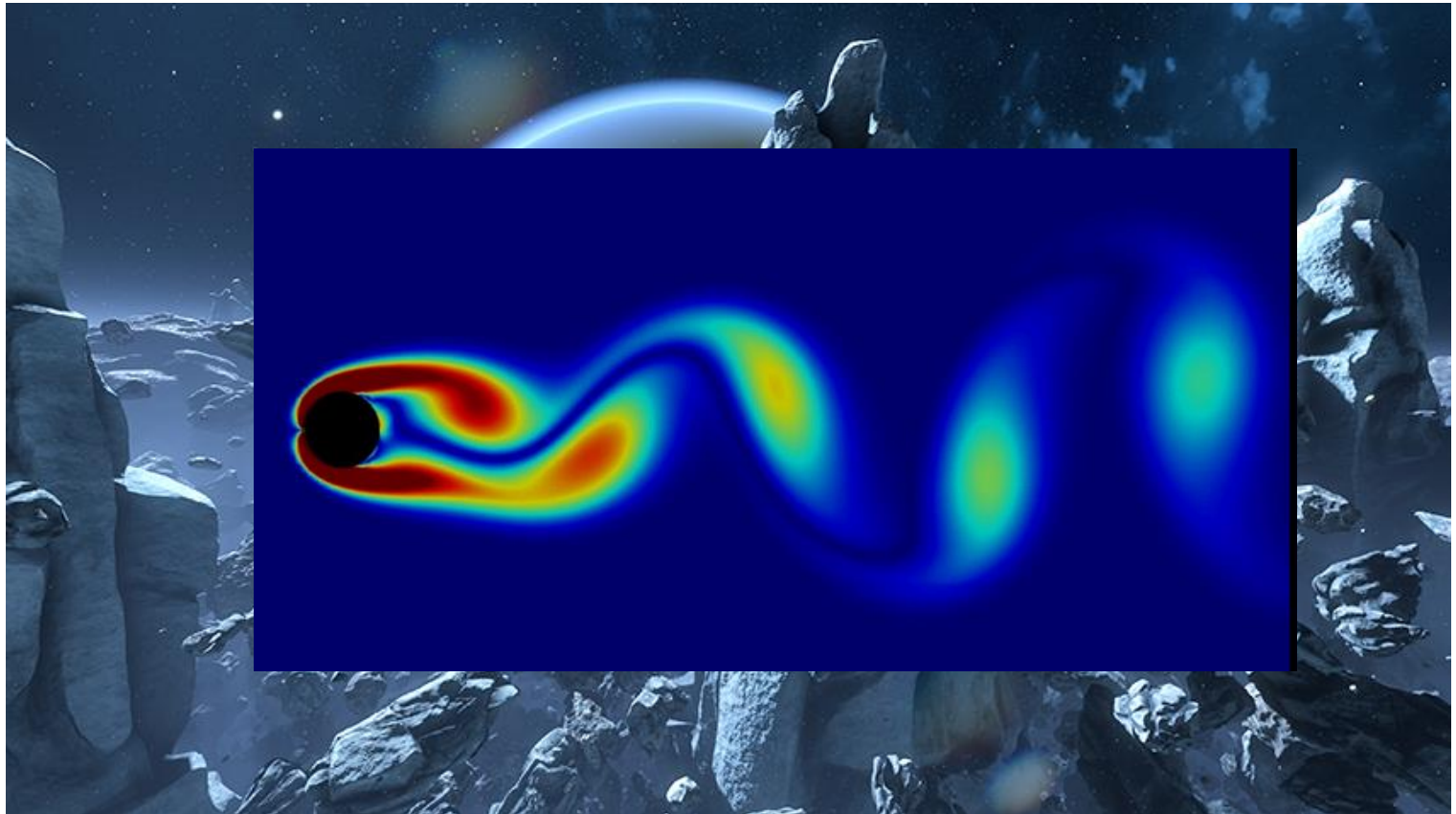
Time for random 10,00,000-d vectors		
dot	2000.0 ms	
dot_jit	59.6 ms	(first call)
		(subsequent)

That was CPU. Numba can also compile to GPU code! But first...

Dec

GPU: Graphics Processing Unit

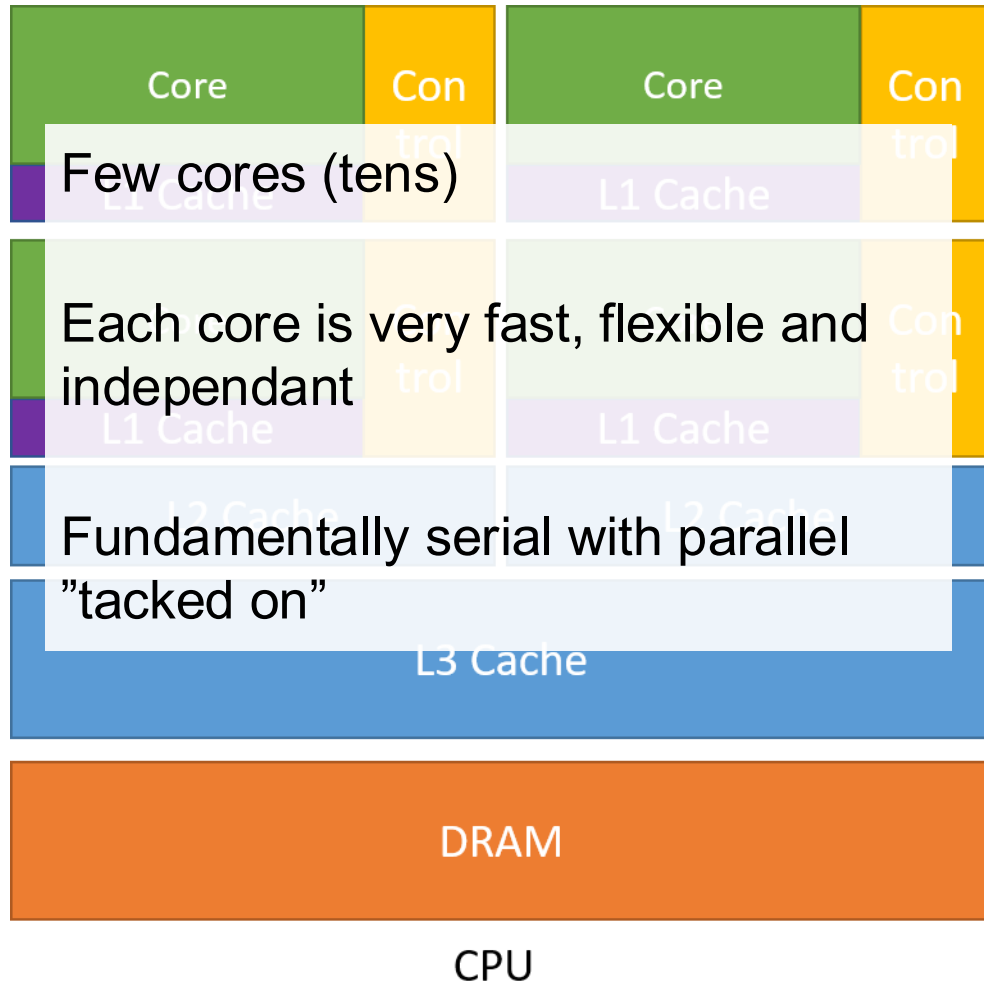
- Originally for graphics tasks, e.g., video games
- Other domains took notice
- Now, GPGPU
General Purpose GPU



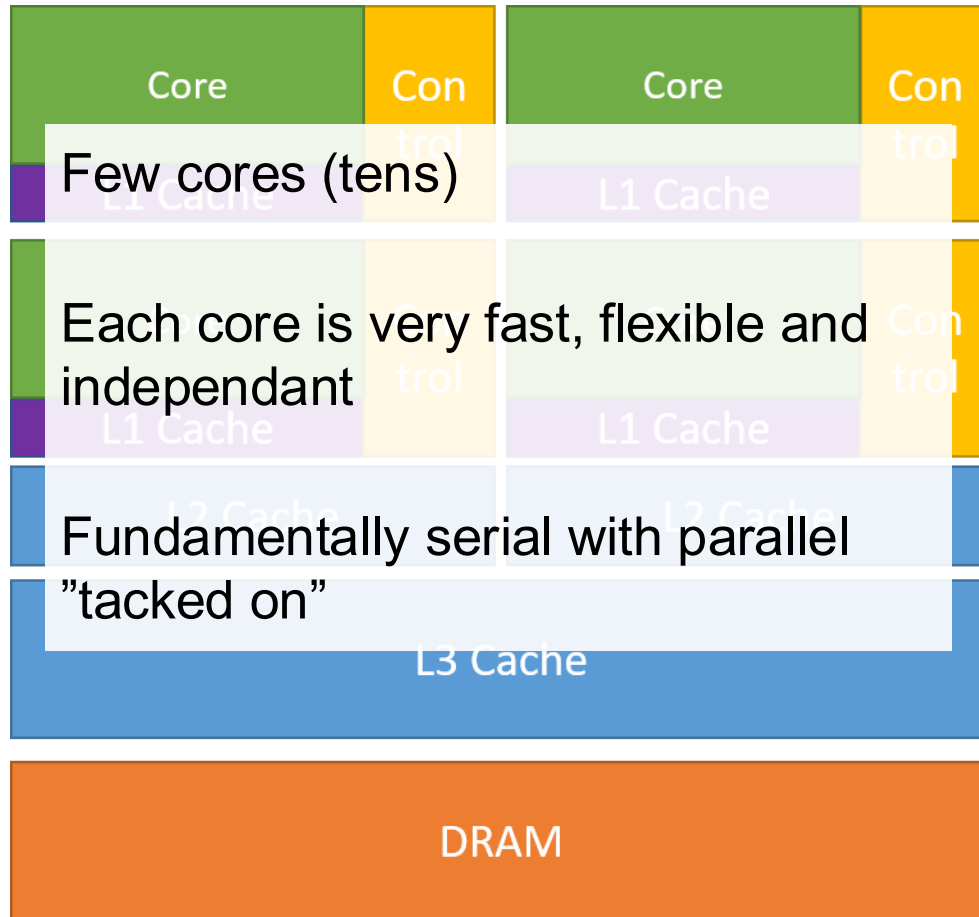


nVIDIA

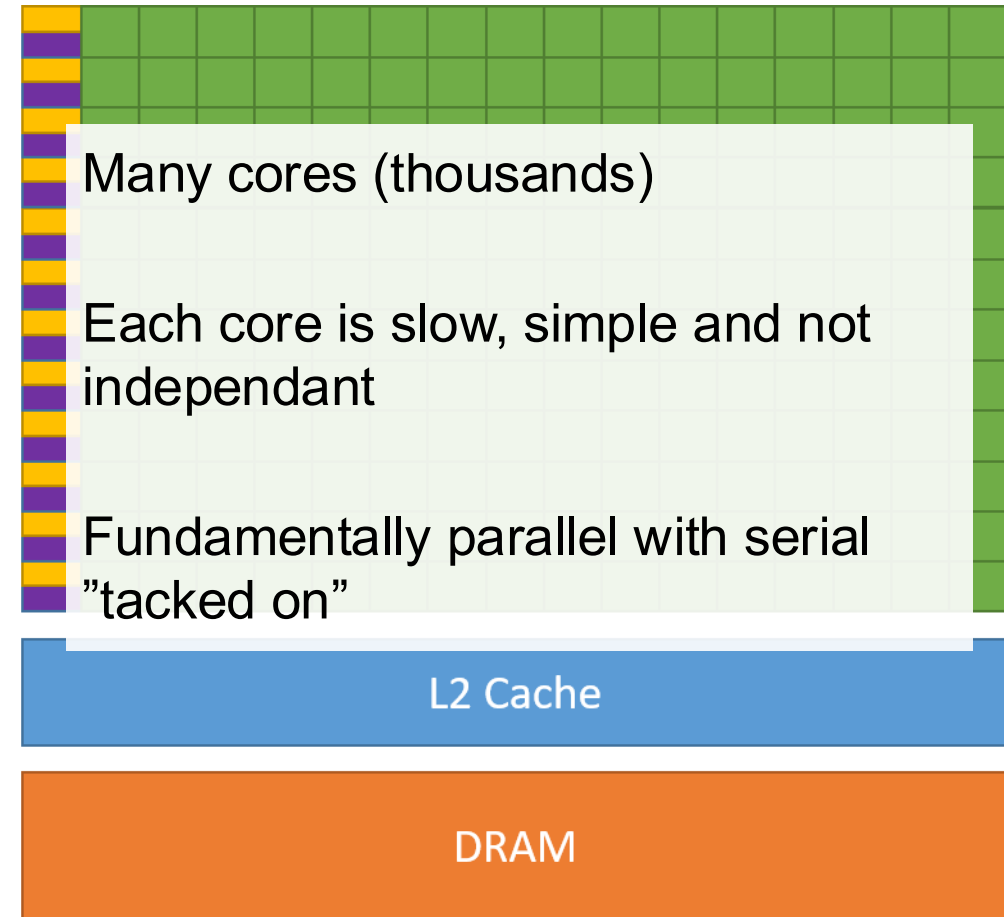
CUDA



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

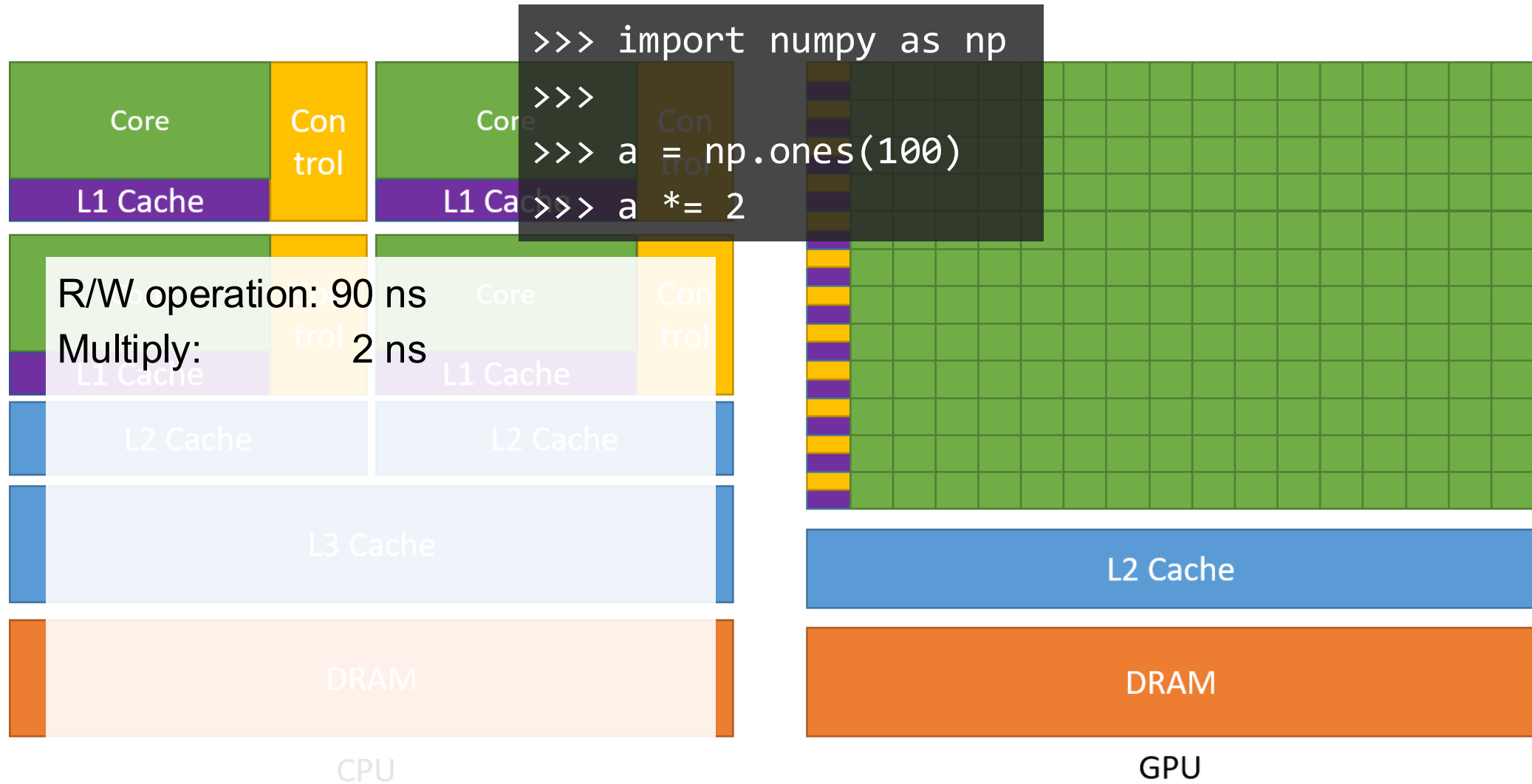


CPU

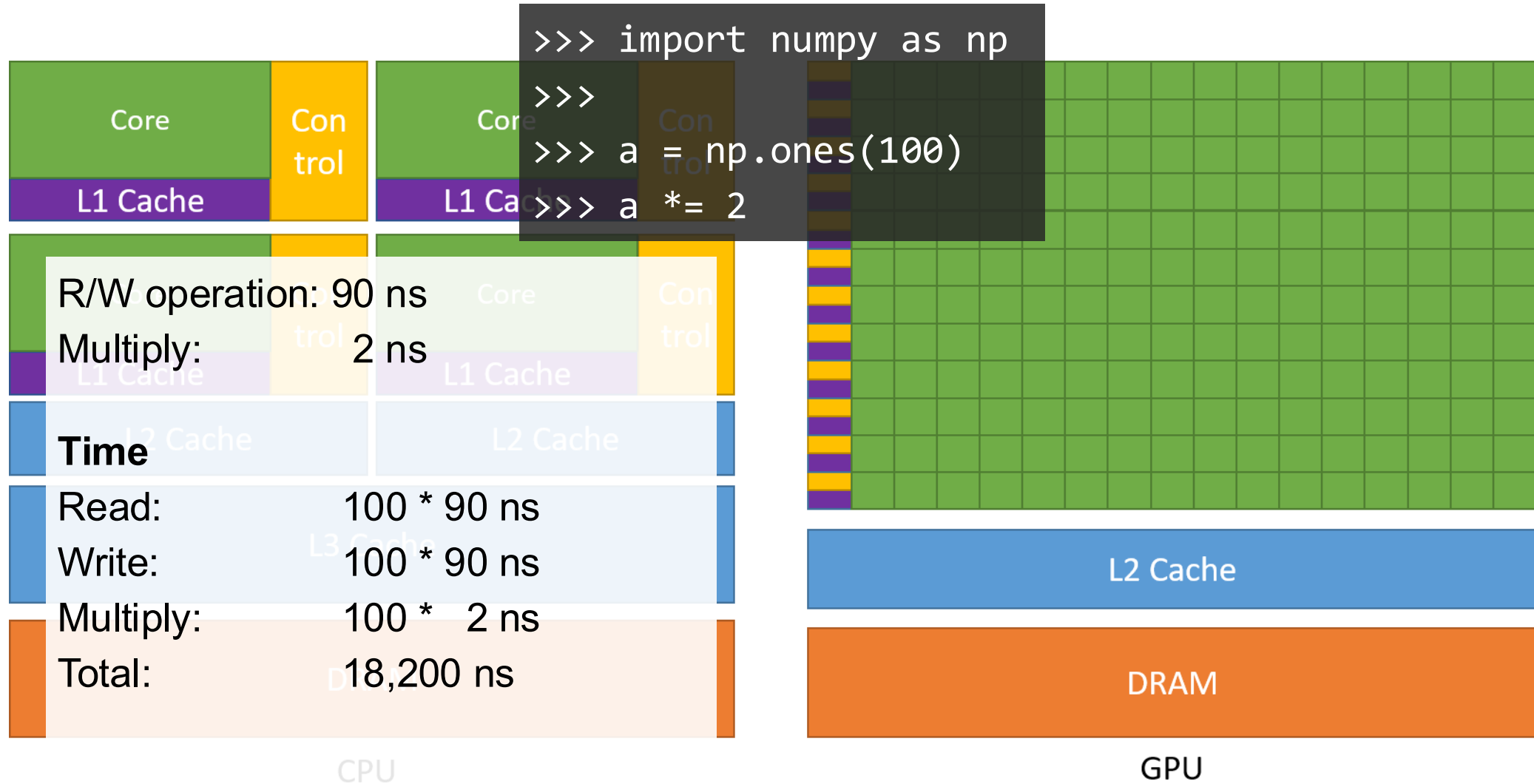


GPU

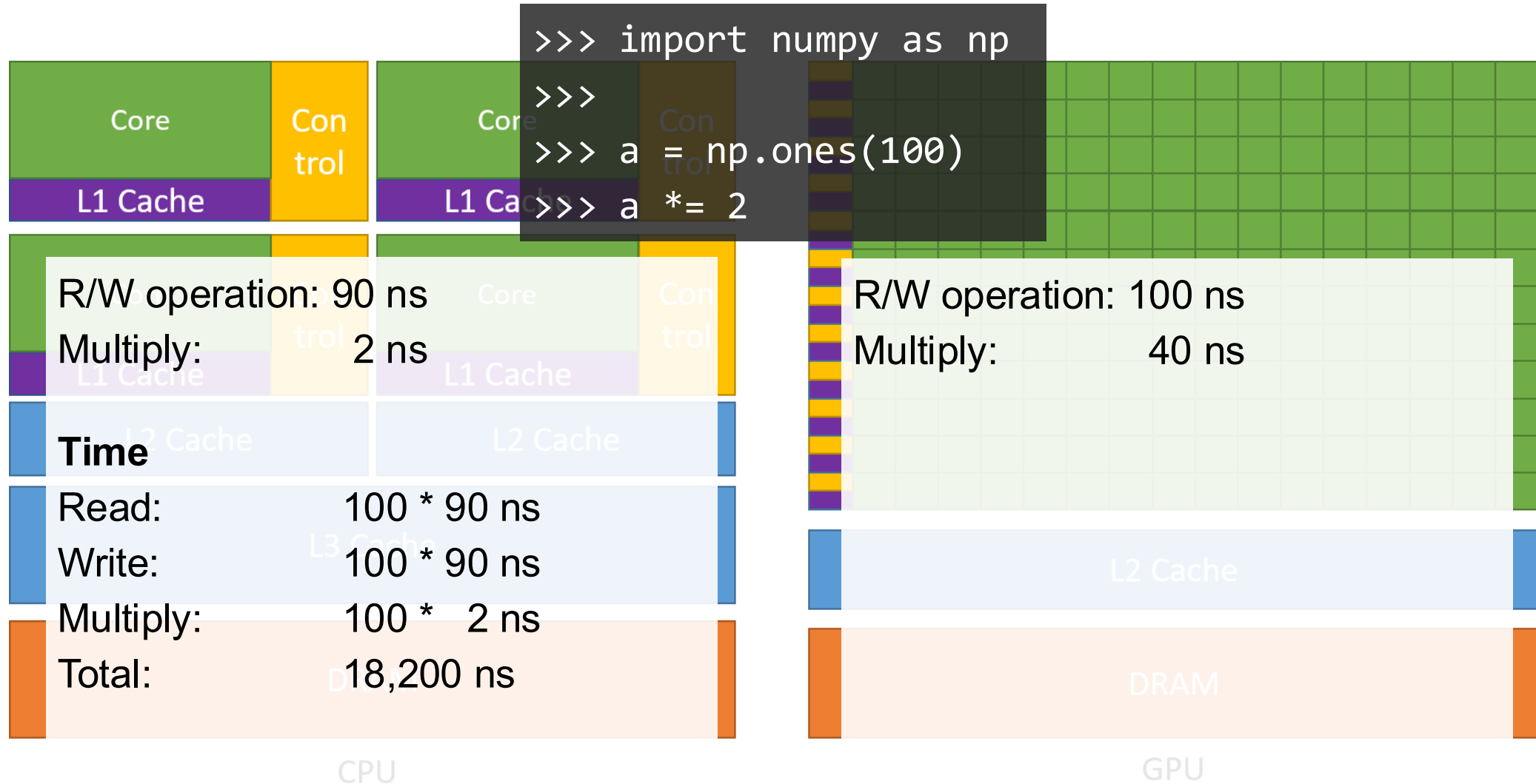
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



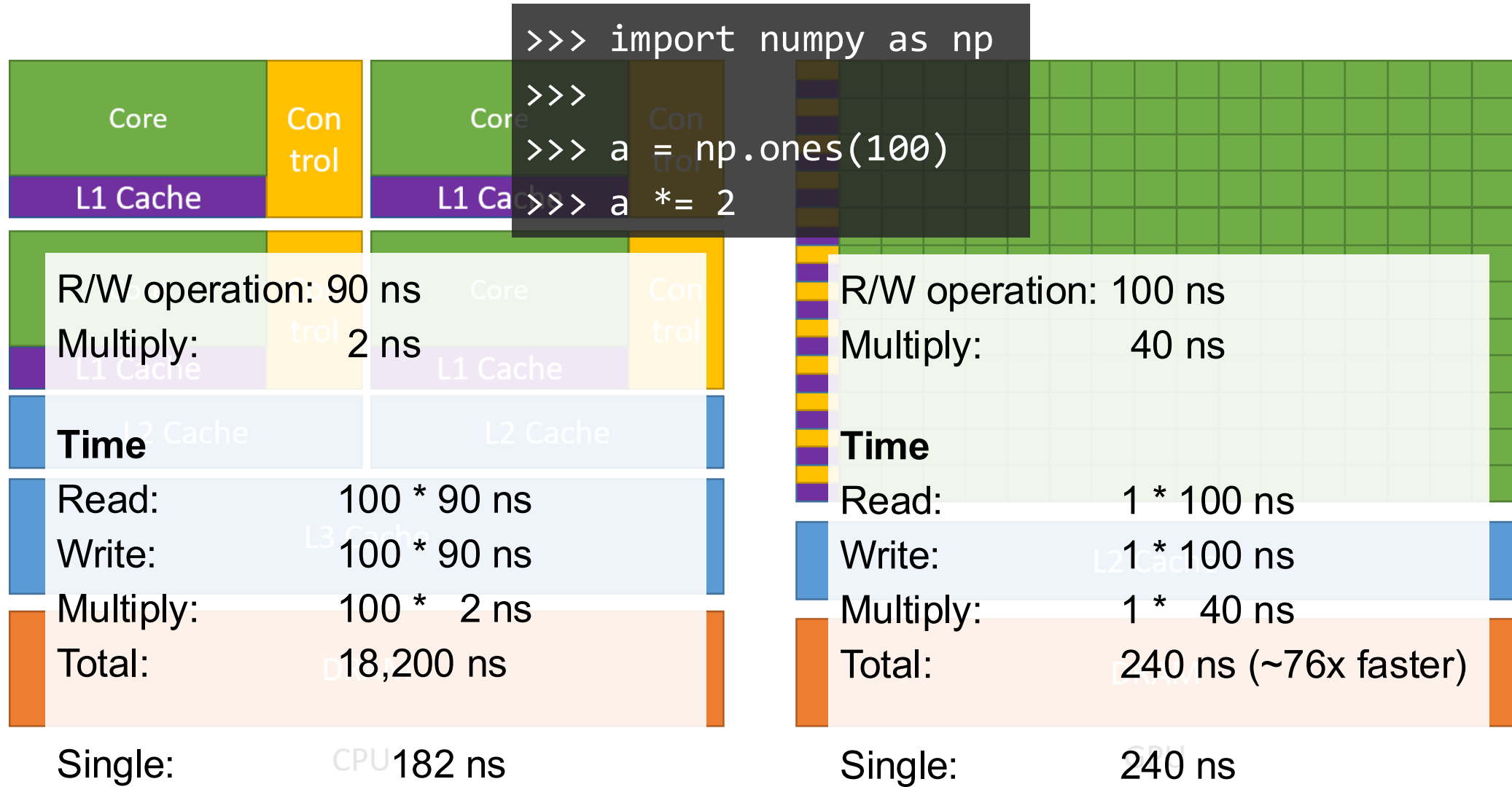
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

CPUs target Latency

Latency = time to complete task

RAM operation: 90 ns



Total: 18,200 ns

Single: CPU 182 ns

GPUs target throughput

Throughput = tasks / time

RAM operation: 100 ns

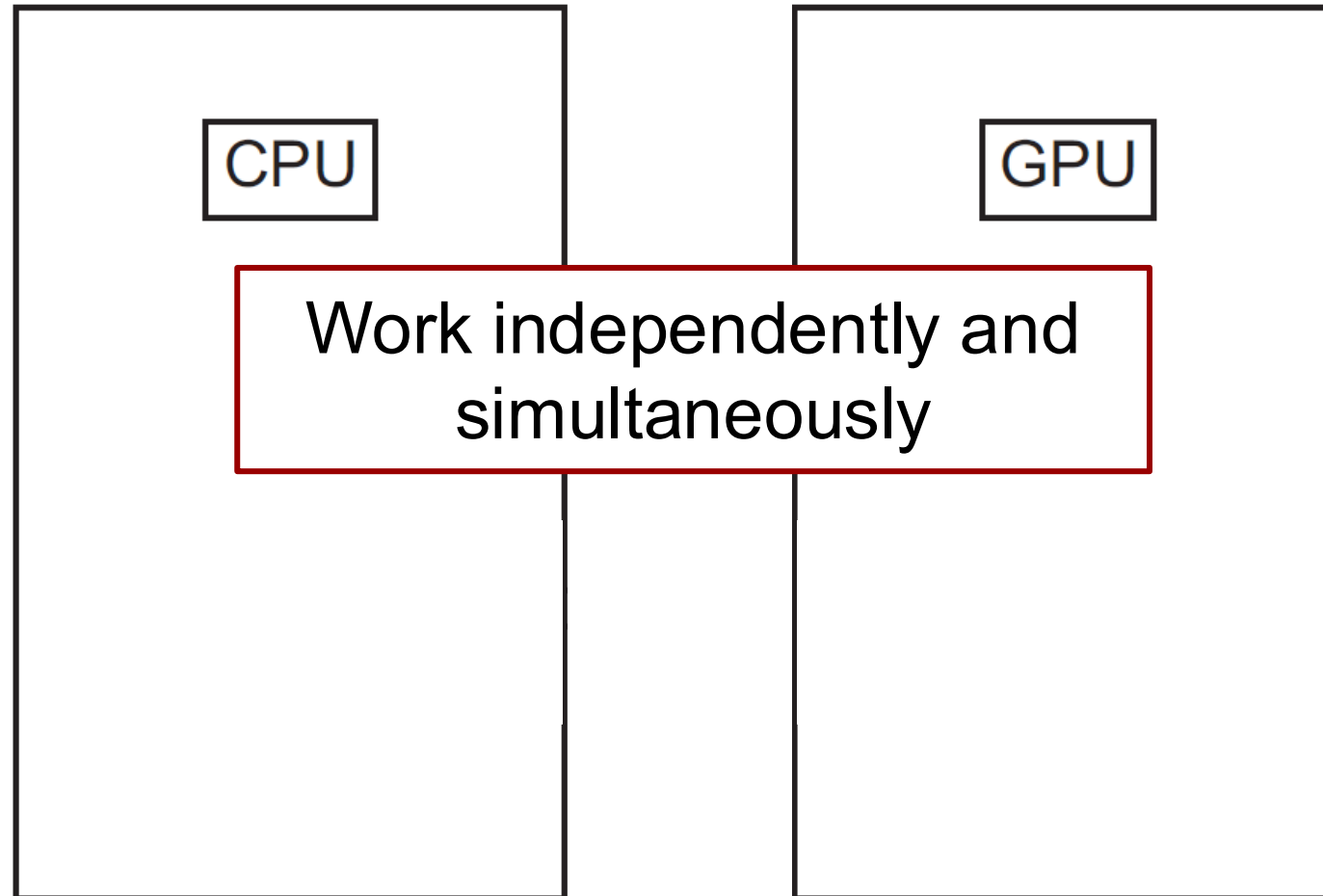


Total: 240 ns (~76x faster)

Single: 240 ns

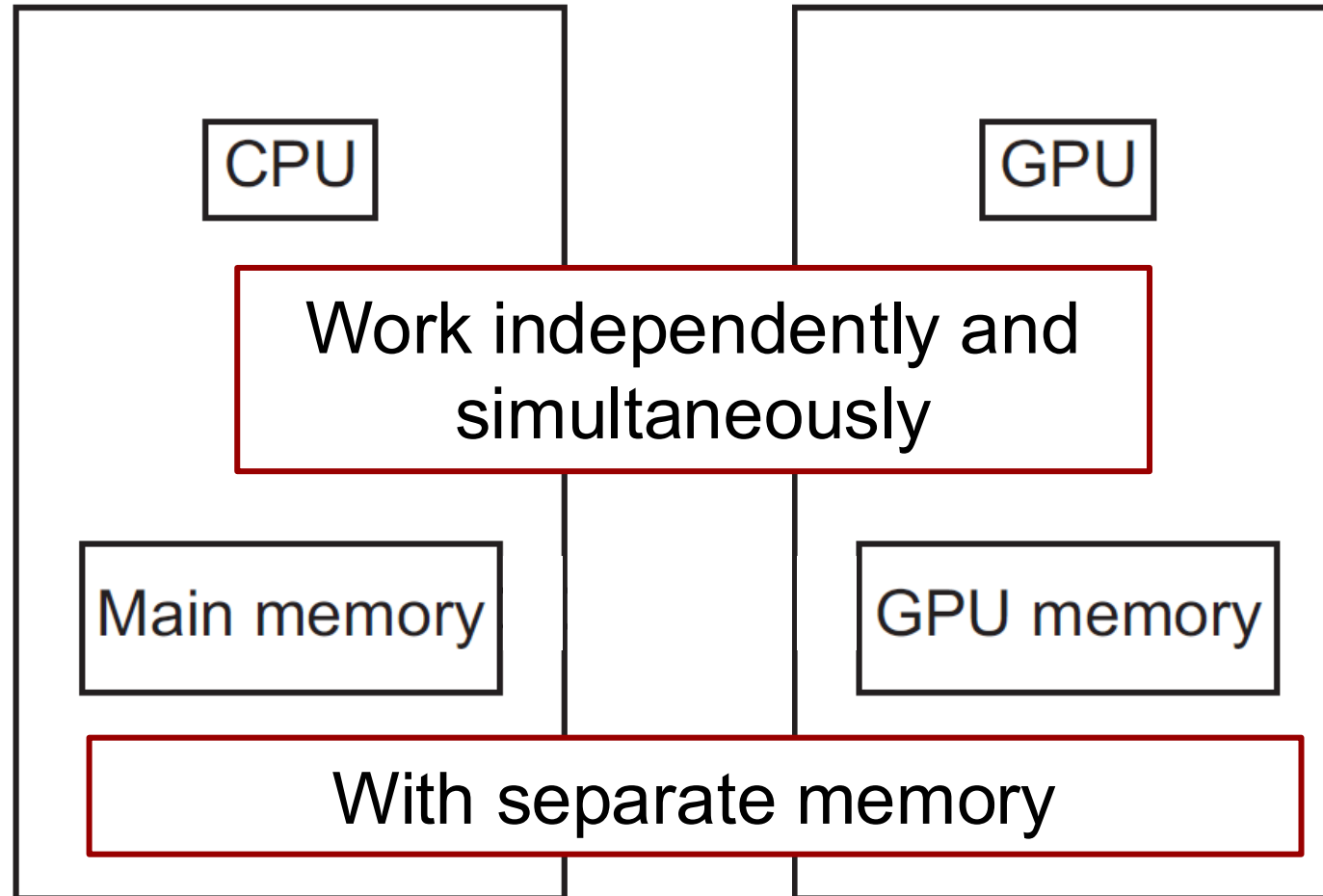
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

GPU are co-processors



Fast Python, Figure 9.1

GPU are co-processors

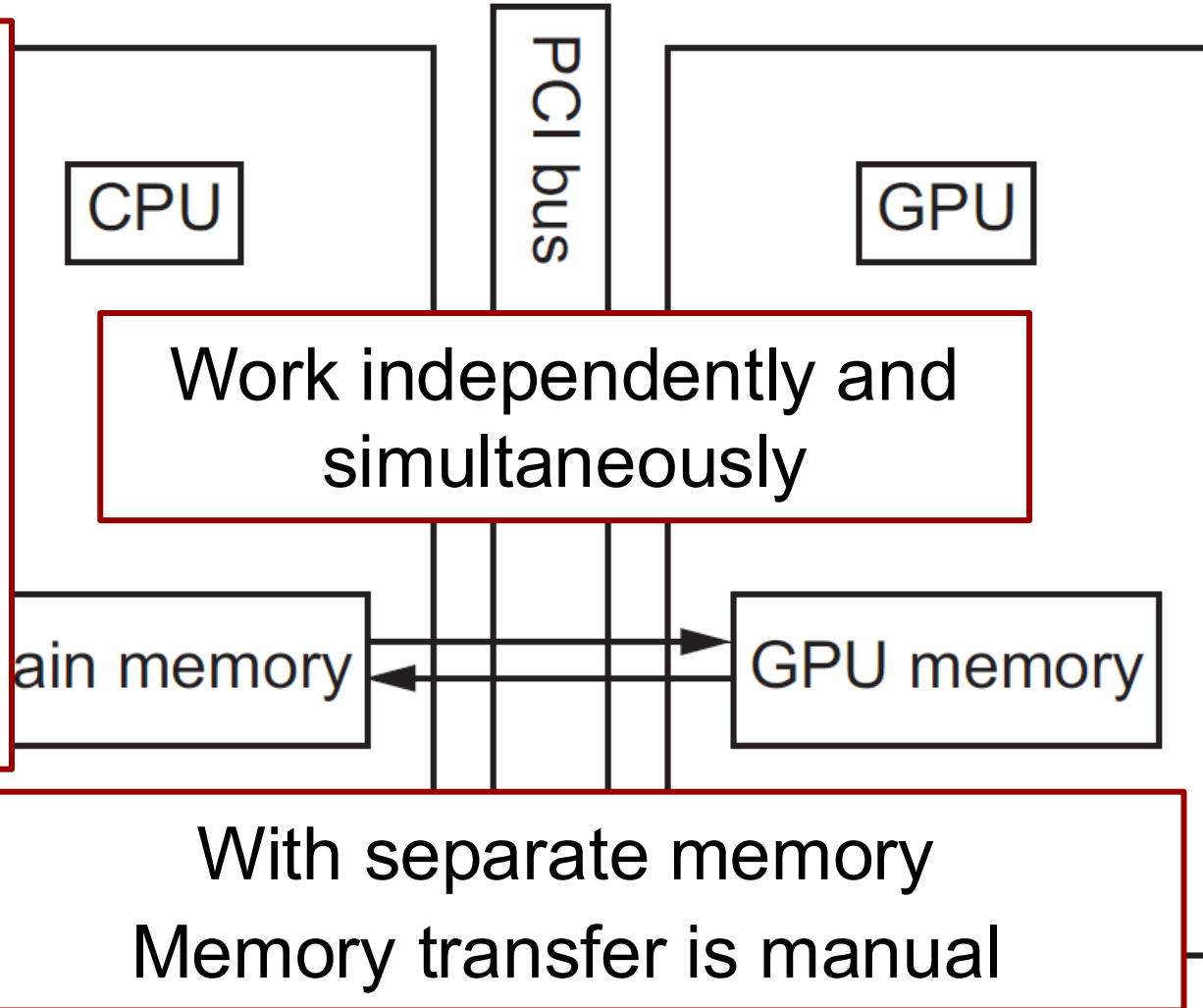


Fast Python, Figure 9.1

GPU are co-processors

CPU takes role of controller:

- Initiates mem transfers
- Schedules computations
- Terms:
CPU = Host
GPU = Device



Fast Python, Figure 9.1

Entry point: GPU kernels

Specifies what each thread should do
When called by CPU: schedules execution on GPU

GPU coordinates execution of threads

Decorator

Special:

- No return value

- Instead, write to output arg

- Should **not** call other kernels!

Typical scenario: map each thread to an output element

```
from numba import cuda
```

Special cuda module

```
@cuda.jit
```

Input

Output

```
def double_kernel(x, y):
```

```
    i = cuda.grid(1)
```

```
    y[i] = 2 * x[i]
```

Ask for assigned thread index

Write result to output

Entry point: GPU kernels

Specifies what each thread should do
When called by CPU: schedules
execution on GPU

GPU coordinates execution of threads

Decorator

Special:

- No return value

- Instead, write to output arg

- Should **not** call other kernels!

Typical scenario: map each thread to
an output element

Special cuda module

```
from numba import cuda
```

Input

Output

```
@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = mul(2, x[i])
```

Ask for assigned thread index

Write result to output

```
@cuda.jit(device=True)
def mul(a, b):
    return a * b
```

Mark as device function

About threads on gpus

Threads are not scheduled (run)
independently

Run in blocks

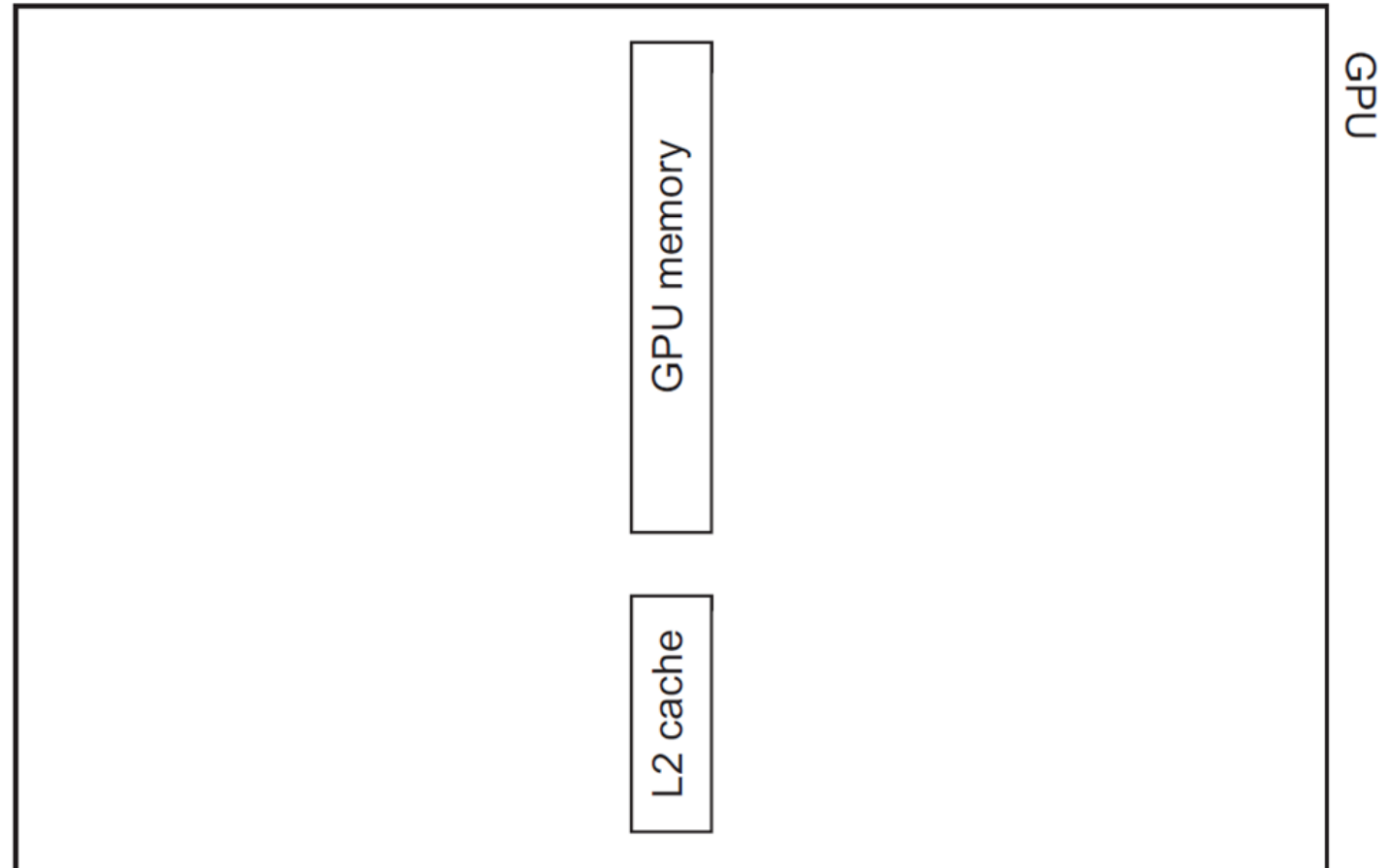
Thread blocks map onto GPU
hardware

About threads

Threads are not scheduled (run) independently

Run in blocks

Thread blocks map onto GPU hardware



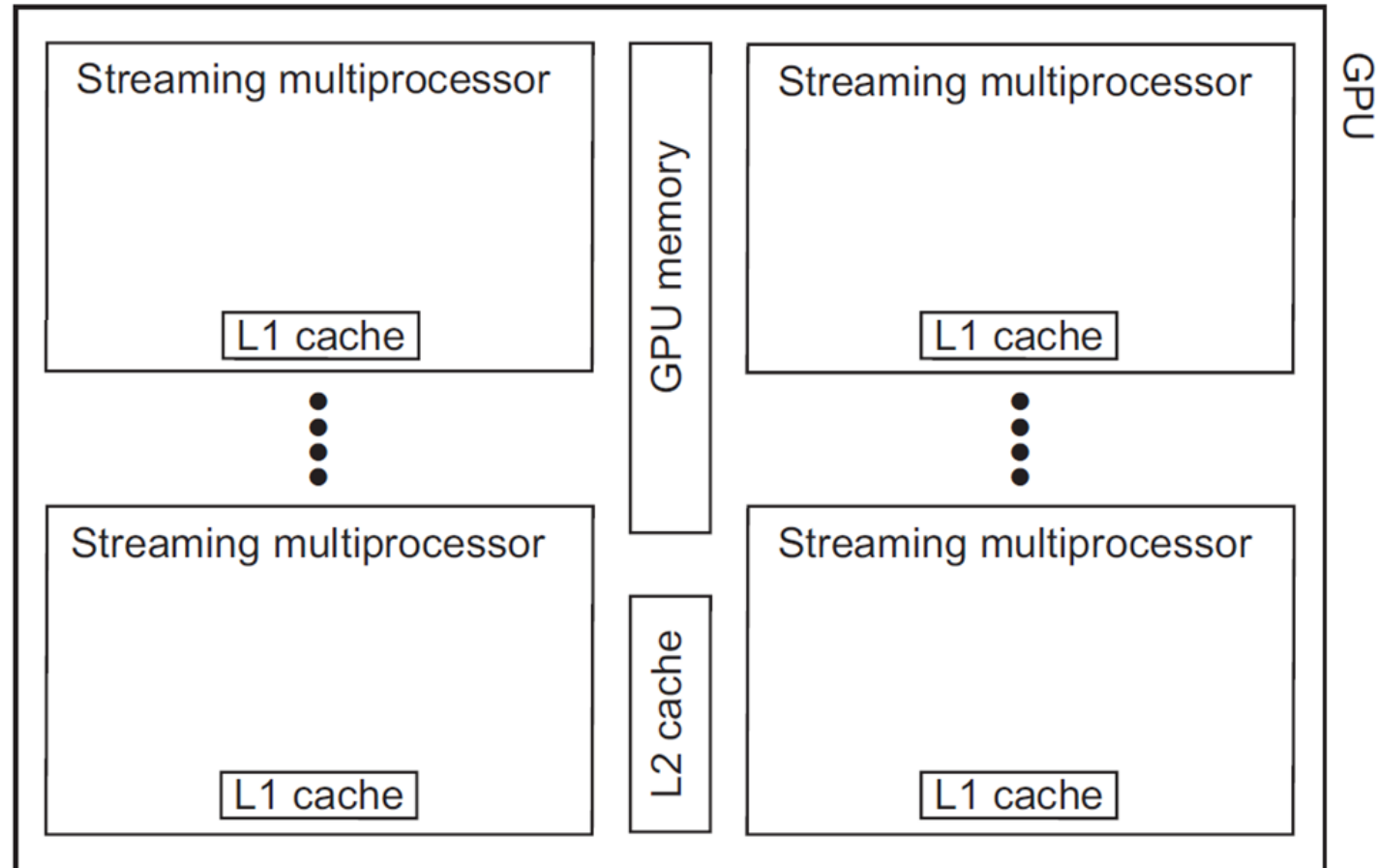
Fast Python, Figure 9.2

About threads

Threads are not scheduled (run) independently

Run in blocks

Thread blocks map onto GPU hardware



Fast Python, Figure 9.2

About threads

Threads are not scheduled (run) independently

Run in blocks

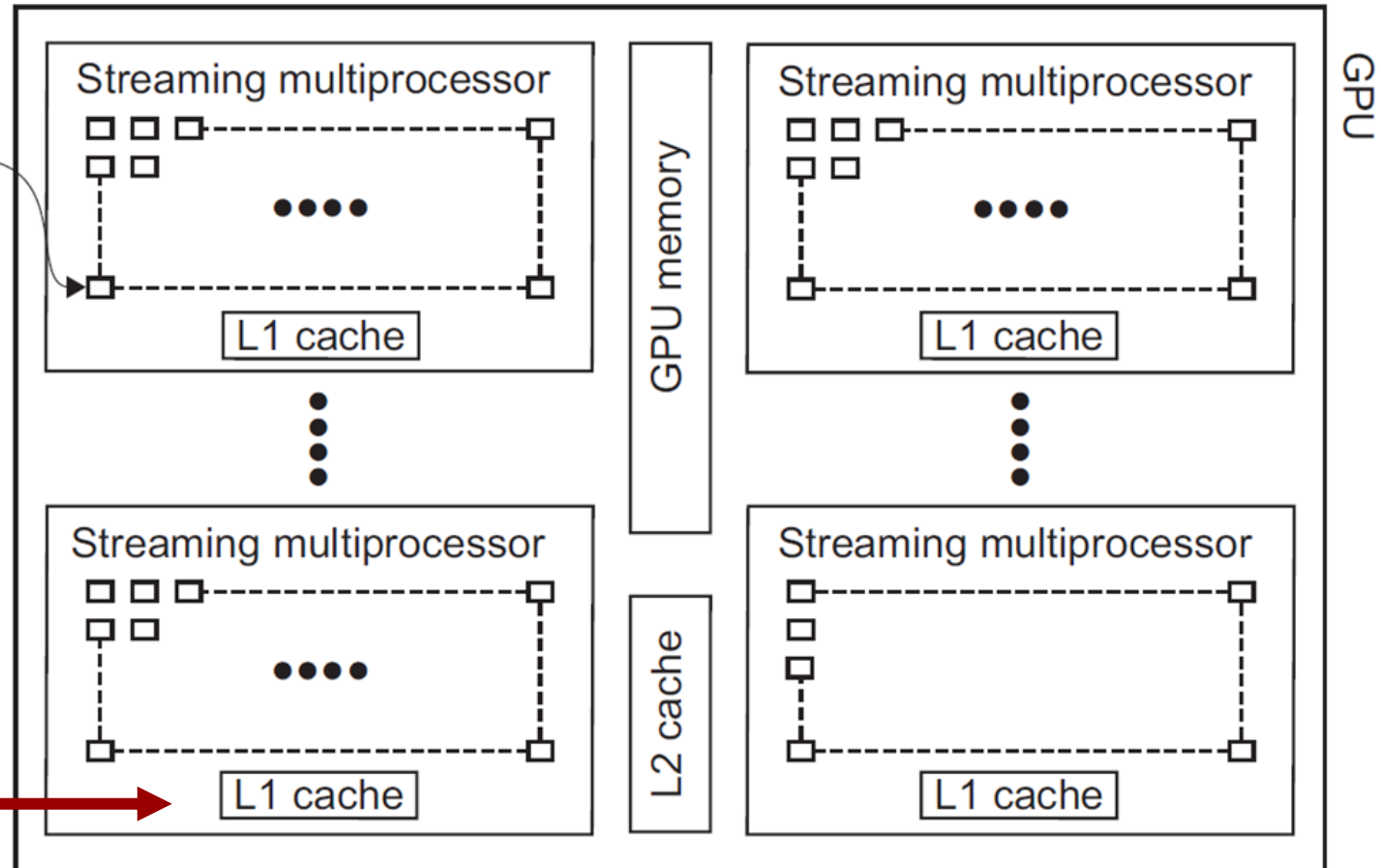
Thread blocks map onto GPU hardware

- Threads map to CUDA cores
- Thread blocks map to SMs

All cores share same L1 cache!
Efficient data sharing in thread block

Tekst Streaming processor (CUDA core)

SM roughly equal a multicore CPU



Fast Python, Figure 9.2

About threads

Who decides the thread blocks?

We do!

Must specify threads per block
and number of blocks

Referred to as the thread grid

Mental model: one thread per output
element. Grid must cover output.

```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
```

```
    i = cuda.grid(1)
```

```
    y[i] = 2 * x[i]
```

```
tpb = 512 # Threads per block
```

```
bpg = len(x) // tpb # Blocks per grid
```

```
double_kernel[bpg, tpb](x, y)
```

Max value 1024

Specify grid

About threads

Who decides the thread blocks?

We do!

Must specify threads per block
and number of blocks

Referred to as the thread grid

Mental model: one thread per output
element. Grid must cover output.

If n not divisible by tpb
need extra block + bounds check

```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
```

```
    i = cuda.grid(1)
```

```
    if i < len(x):
```

```
        y[i] = 2 * x[i]
```

```
def get_bpg(n, tpb):
```

```
    return (n + (tpb - 1)) // tpb
```

```
tpb = 512
```

```
bpg = get_bpg(len(x), tpb)
```

```
double_kernel[bpg, tpb](x, y)
```

Specify grid



About threads

Can also get more detailed index of thread

Useful for advanced algos.

Index of thread in block

Index of thread block

Threads per block

```
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.blockIdx.x * cuda.blockDim.x \
        + cuda.threadIdx.x
    if i < len(x):
        y[i] = 2 * x[i]

def get_bpg(n, tpb):
    return (n + (tpb - 1)) // tpb

tpb = 512
bpg = get_bpg(len(x), tpb)
double_kernel[bpg, tpb](x, y)
```

About threads

Can also have 2D (and 3D) thread grids.

Useful for when domain is 2D or 3D

Must specify thread grid as tuples

Can also use detailed indexes:

`cuda.threadIdx.[x,y,z]`

`cuda.blockIdx.[x,y,z]`

`cuda.blockDim.[x,y,z]`

```
from numba import cuda

@cuda.jit
def double_kernel_2d(x, y): 2D thread index
    i, j = cuda.grid(2)
    if i < x.shape[0] and j < x.shape[1]:
        y[i, j] = 2 * x[i, j]

tpb = 16, 16
bpg = (x.shape[0] // tpb[0],
       x.shape[1] // tpb[1])
double_kernel_2d[bpg, tpb](x, y)
```

Quiz time!

That was the stuff from the book
I have two more topics

Warps

Memory transfers

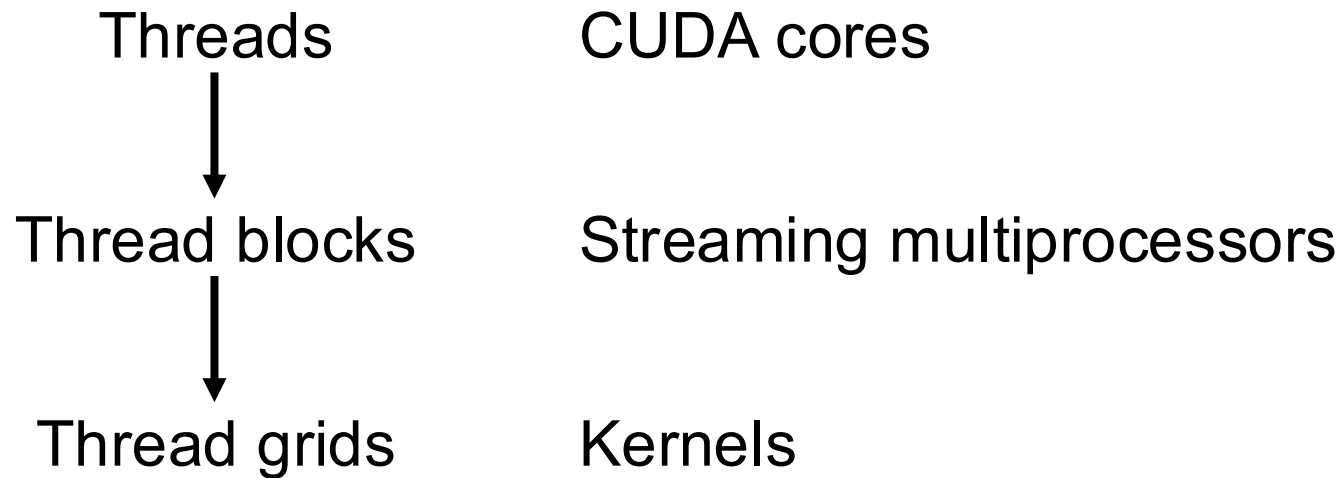
Fundamental for:

Functionality (how does GPU coding work)

High performance

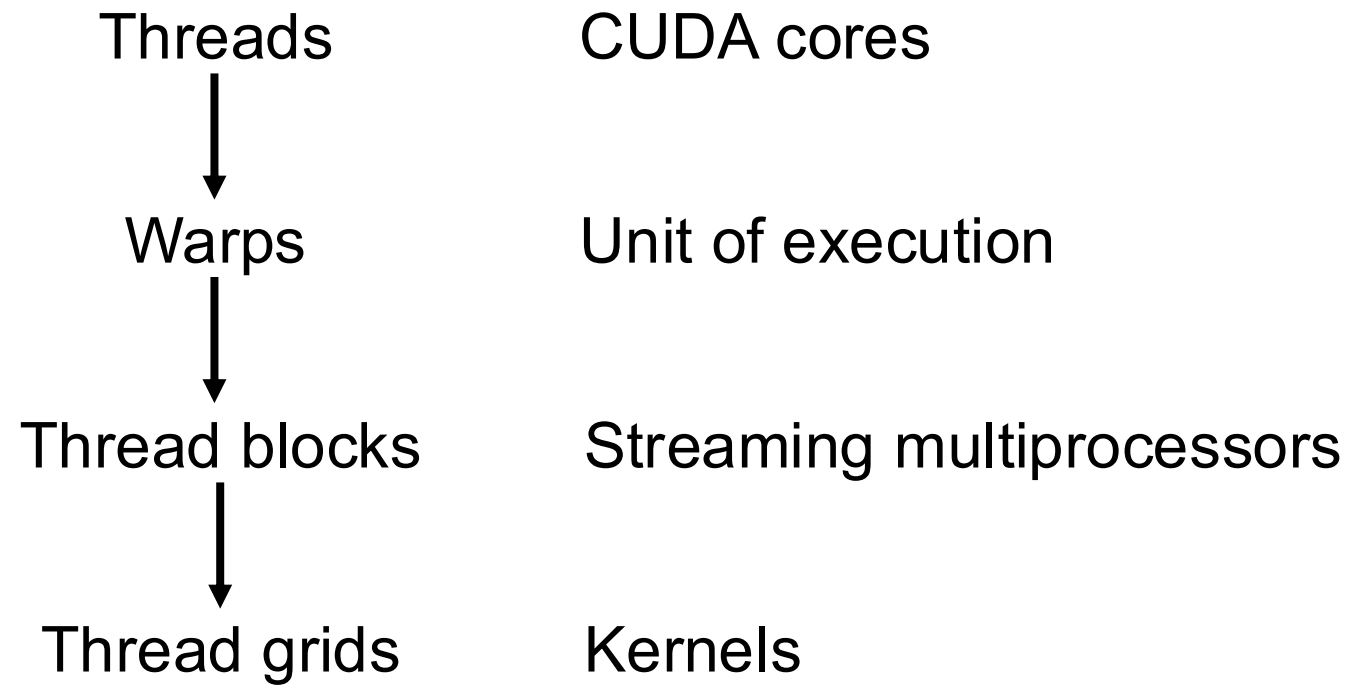
Warps

Hierarchy of execution

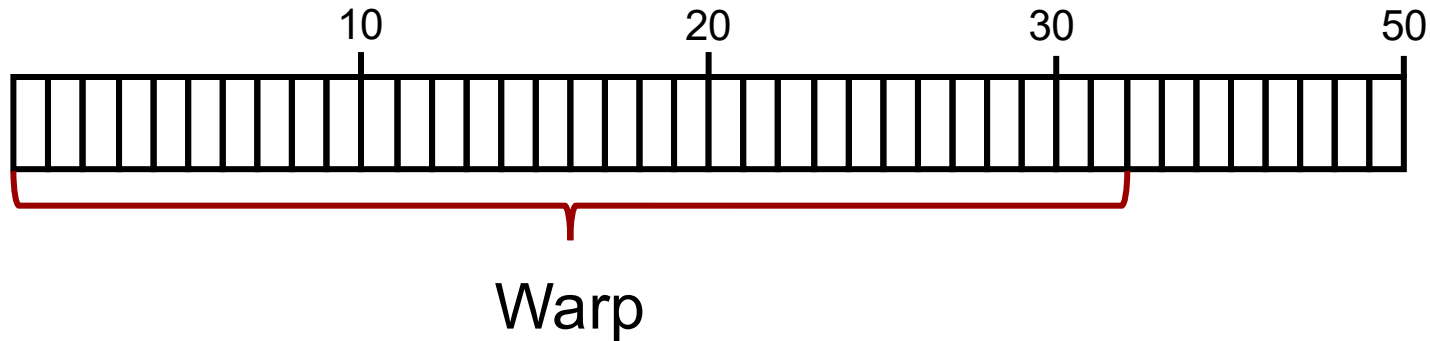


Warps

Hierarchy of execution



Warps



Threads does **not** execute operations independently

Operations are grouped in *warps* of 32 threads

All threads in a warp execute *the same operation*

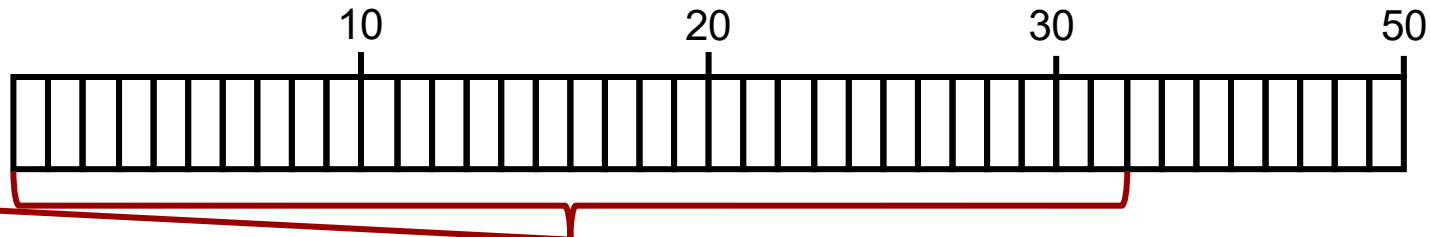
Multiple warps needed when threads diverge

Warp determined by thread ID (i.e., `cuda.grid(i)`)

```
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]
```

Warps



```
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
```

Very different from CPU where each core runs completely independently!

Operations are grouped in *warps* of 32 threads

All threads in a warp execute *the same operation*

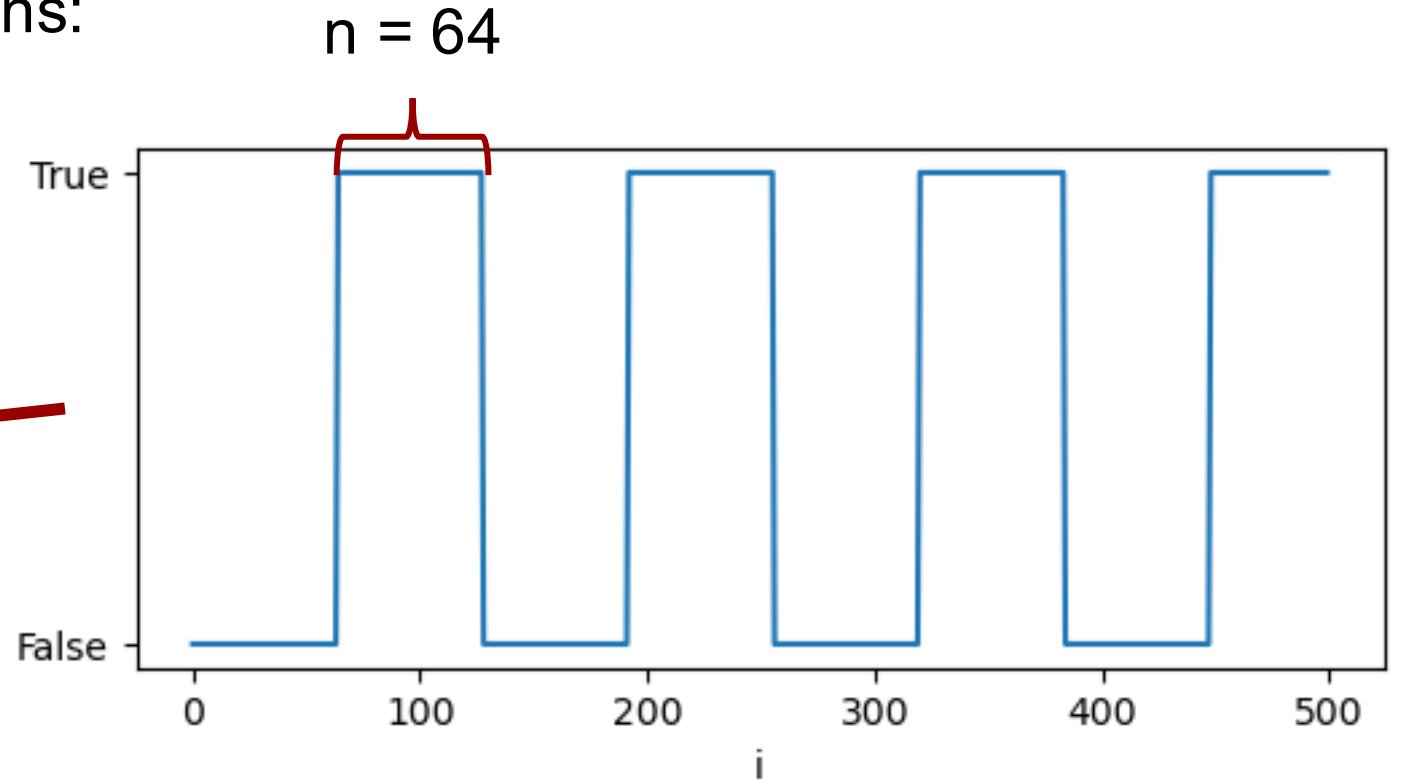
Multiple warps needed when threads diverge

Warp determined by thread ID (i.e., `cuda.grid(i)`)

Warps

This has performance implications:

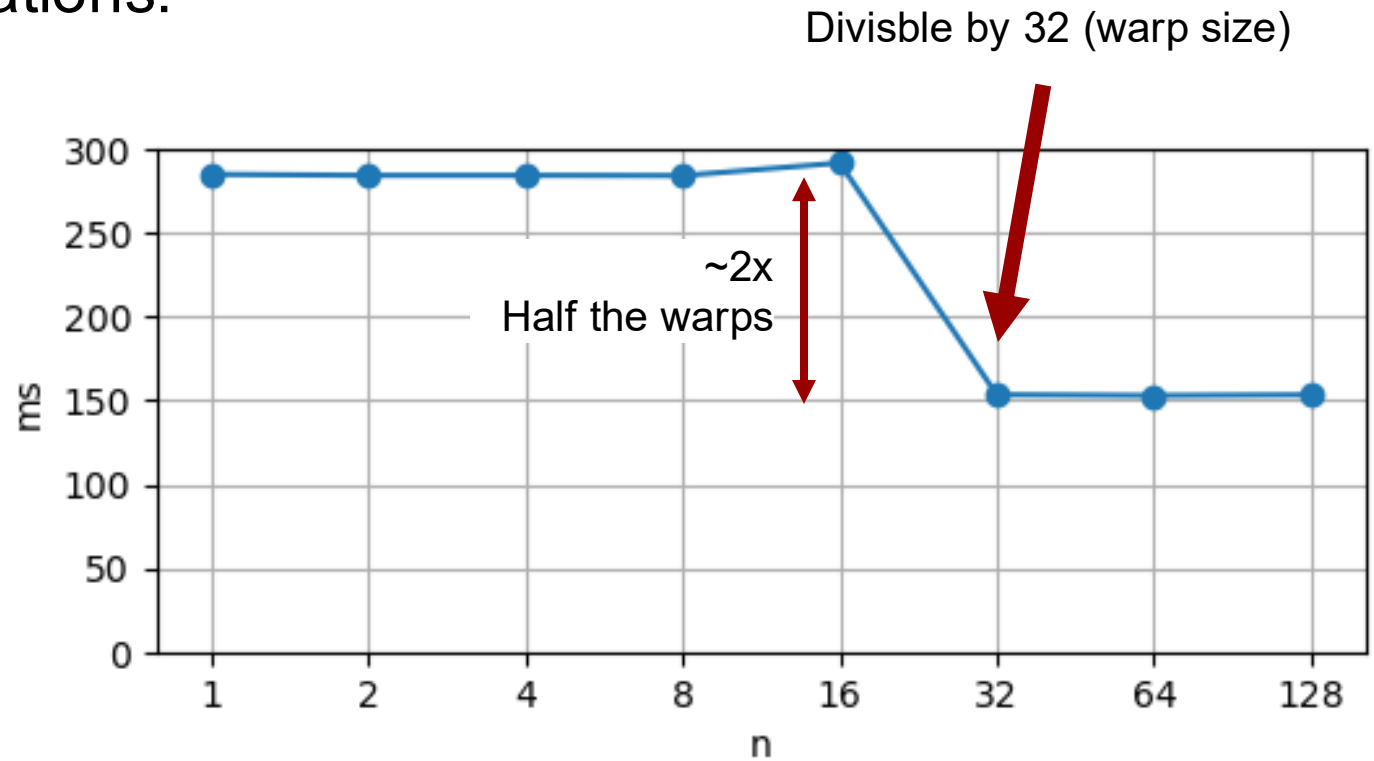
```
@cuda.jit
def condscale(x, y, n):
    i = cuda.grid(1)
    for _ in range(100):
        if (i // n) % 2 == 0:
            y[i] = x[i] * 3
        else:
            y[i] = x[i] / 3
```



Warps

This has performance implications:

```
@cuda.jit
def condscale(x, y, n):
    i = cuda.grid(1)
    for _ in range(100):
        if (i // n) % 2 == 0:
            y[i] = x[i] * 3
        else:
            y[i] = x[i] / 3
```



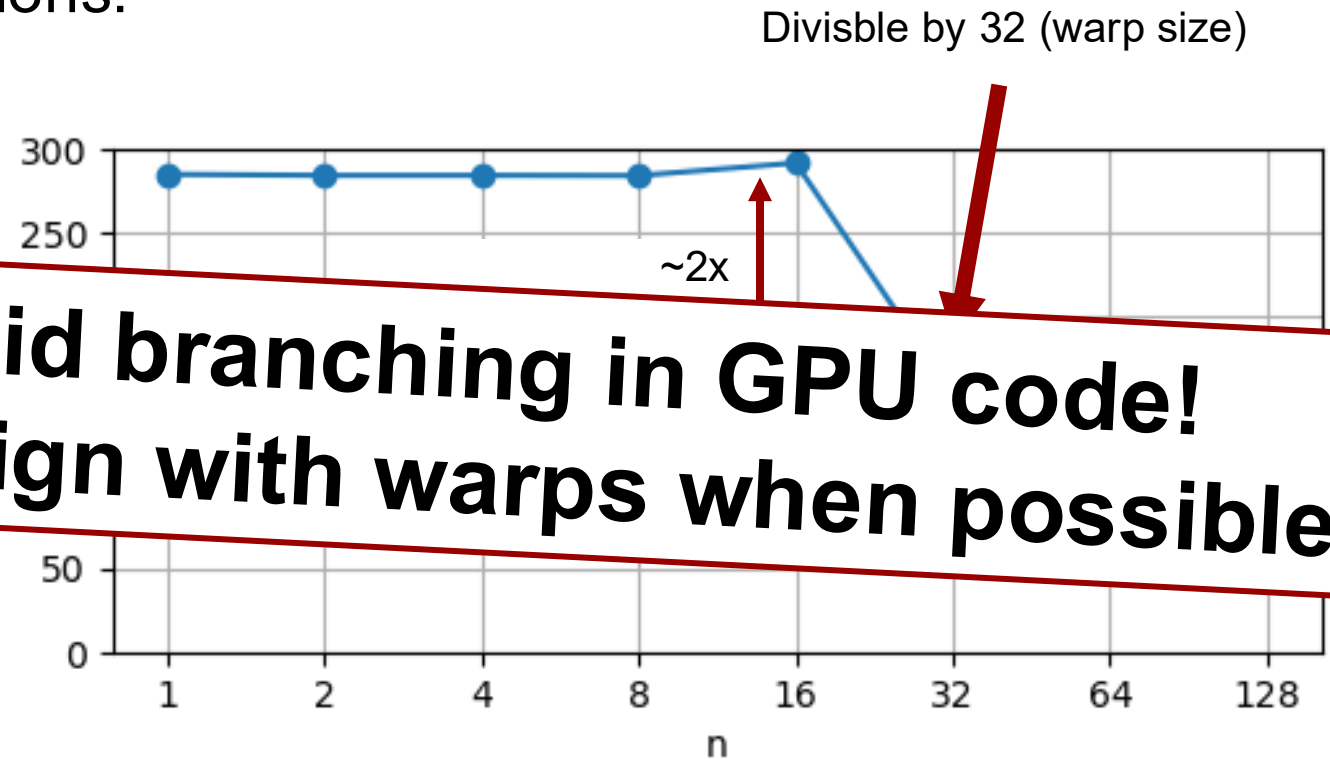
Warps

This has performance implications:

```
@cuda.jit
def condscale(x, y, n):
```

Moral: try to avoid branching in GPU code!
If you must, then align with warps when possible

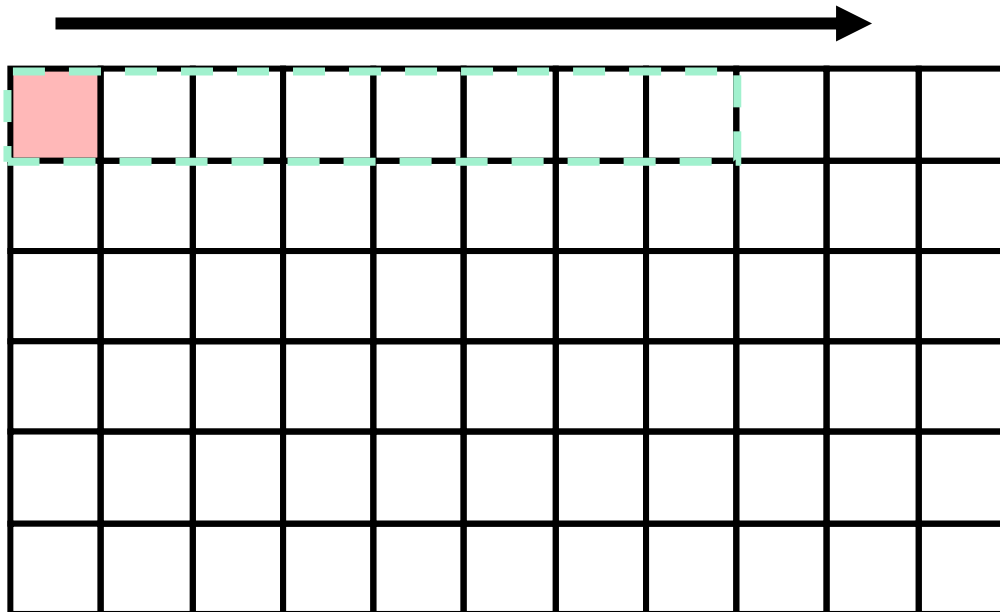
```
    else:
        y[i] = x[i] / 3
```



Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed



L1 cache line on GPU: 128 bytes

Assuming float32 (4 bytes)

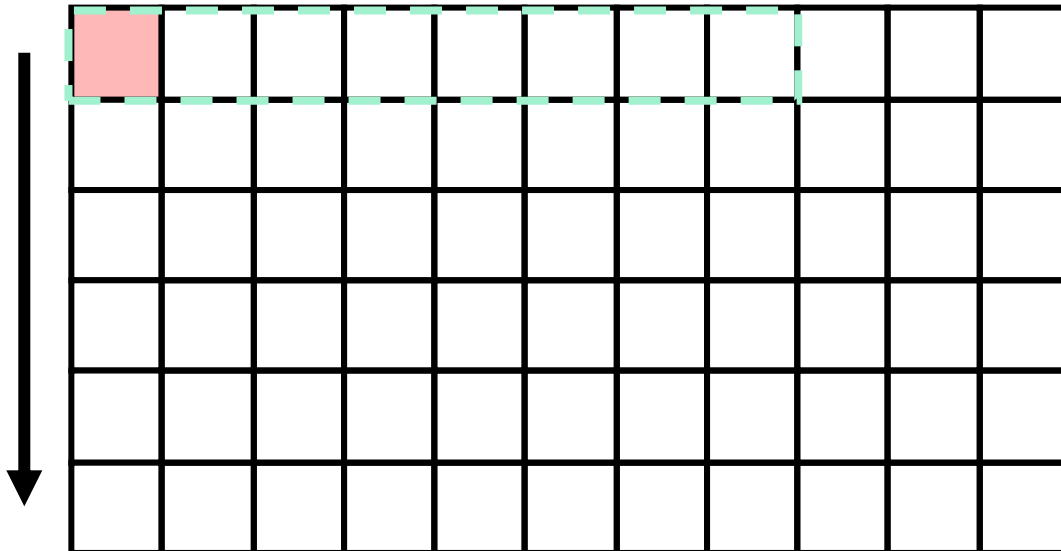
1 cache line = 32 numbers

Only 1 load for the whole warp!

Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed



L1 cache line on GPU: 128 bytes

Assuming float32 (4 bytes)

1 cache line = 32 numbers

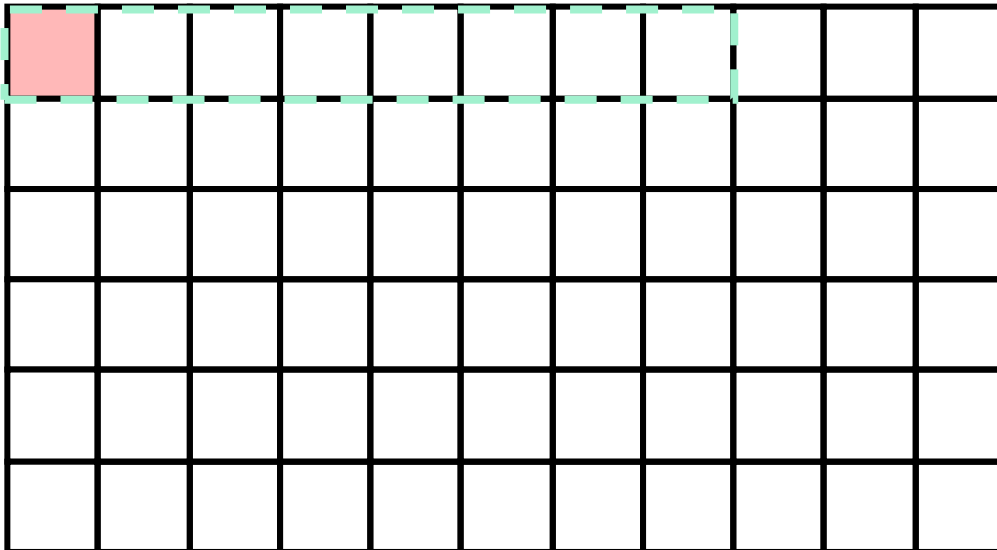
No thread shares cache line

32 loads

Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed



```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel_2d(x, y):
```

```
    i, j = cuda.grid(2)
```

```
    y[i, j] = 2 * x[i, j]
```

```
tpb = 32, 32
```

```
bpg = (x.shape[0] // tpb[0],
```

```
        x.shape[1] // tpb[1])
```

```
double_kernel_2d[bpg, tpb](x, y)
```

Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed

Warps are based on thread ID

2D block of size (Dx, Dy):

$$\text{ID} = x + y * \text{Dx}$$

3D block of size(Dx, Dy, Dz):

$$\text{ID} = x + y * \text{Dx} + z * \text{Dx} * \text{Dy}$$

Column-wise layout!

```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel_2d(x, y):
```

```
    i, j = cuda.grid(2)
```

```
    y[i, j] = 2 * x[i, j]
```

```
tpb = 32, 32
```

```
bpg = (x.shape[0] // tpb[0],
```

```
        x.shape[1] // tpb[1])
```

```
double_kernel_2d[bpg, tpb](x, y)
```

Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed

Warps are based on thread ID

2D block of size (Dx, Dy):

$$\text{ID} = x + y * \text{Dx}$$

3D block of size(Dx, Dy, Dz):

$$\text{ID} = x + y * \text{Dx} + z * \text{Dx} * \text{Dy}$$

Column-wise layout!

```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel_2d(x, y):
```

```
    j, i = cuda.grid(2)
```

```
    y[i, j] = 2 * x[i, j]
```

```
tpb = 32, 32
```

```
bpg = (x.shape[1] // tpb[0],
```

```
        x.shape[0] // tpb[1])
```

```
double_kernel_2d[bpg, tpb](x, y)
```

Time for 4096 x 4096 array:

i, j = cuda.grid(2) 0.64 ms

j, i = cuda.grid(2) 0.19 ms

Warps

Thread blocks share same L1 cache

If threads in a warp read data from same cache line, only one load is performed

Moral: try to setup code *and* compute grid so warps access data cache efficiently

$$\text{ID} = x + y * \text{Dx}$$

3D block of size(Dx, Dy, Dz):

$$\text{ID} = x + y * \text{Dx} + z * \text{Dx} * \text{Dy}$$

Column-wise layout!

```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel_2d(x, y):
```

```
    j, i = cuda.grid(2)
```

```
    x.shape[0] // tpb[1])
```

```
double_kernel_2d[bpg, tpb](x, y)
```

Time for 4096 x 4096 array:

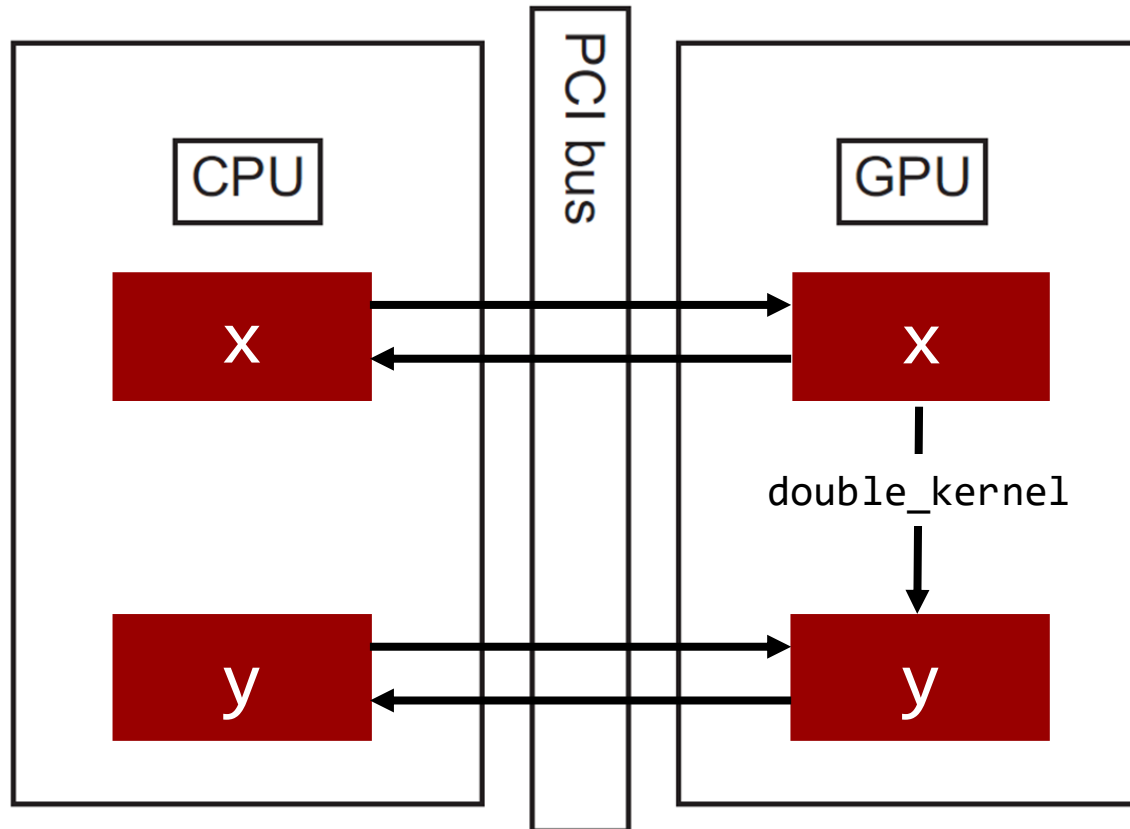
i, j = cuda.grid(2) 0.64 ms

j, i = cuda.grid(2) 0.19 ms

Memory transfers

How does this work?

Numba's default:



```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]
```

CPU

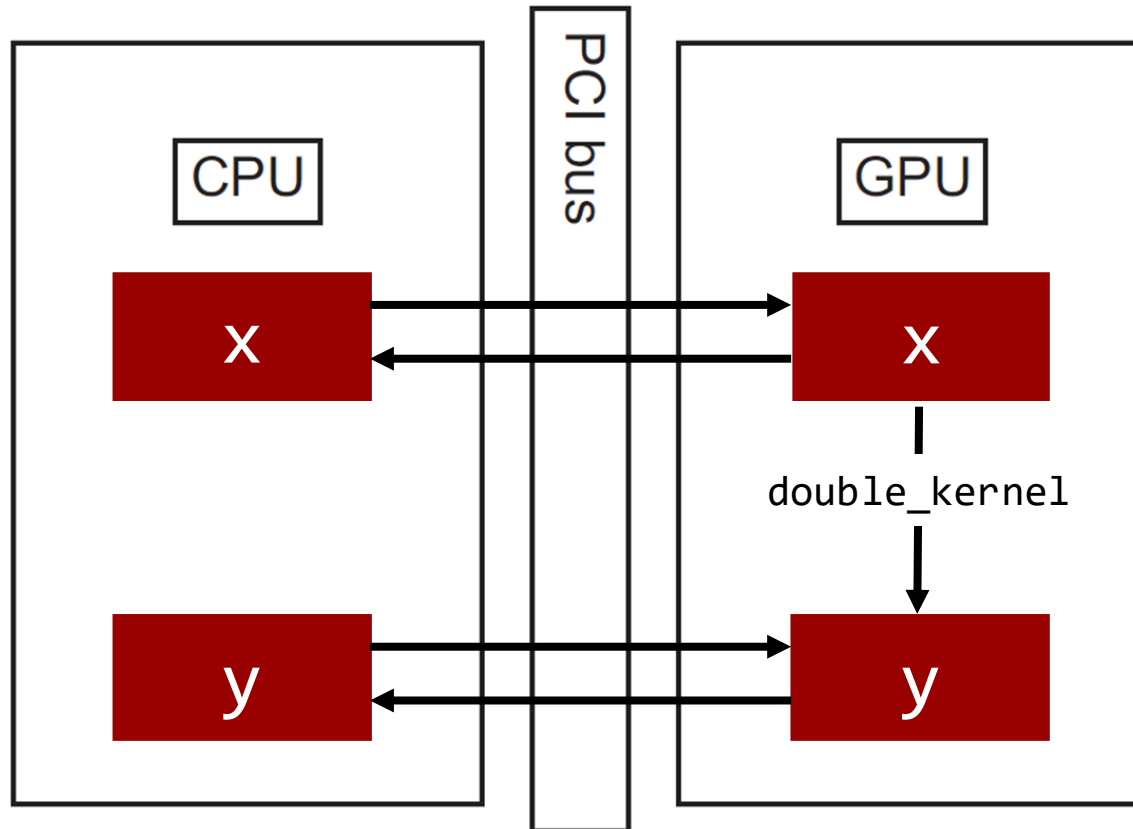
```
x = np.random.rand(8 * 512)
y = np.empty_like(x)
```

```
tpb = 512
bpg = len(x) // tpb
double_kernel[bpg, tpb](x, y)
```

GPU

Memory transfers

What if we want to run multiple kernels?
E.g., for synchronization



```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
    i = cuda.grid(1)
    y[i] = 2 * x[i]
```

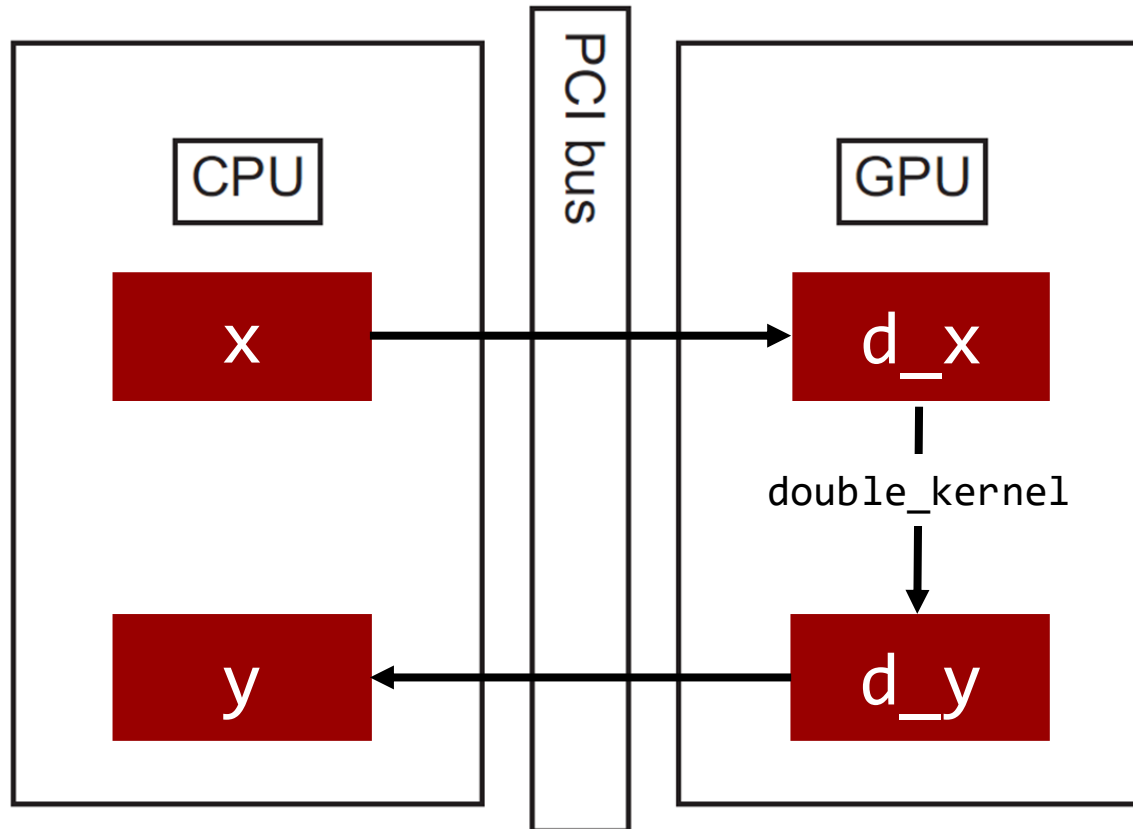
```
x = np.random.rand(8 * 512)
y = np.empty_like(x)
```

```
tpb = 512
bpg = len(x) // tpb
double_kernel[bpg, tpb](x, y)
double_kernel[bpg, tpb](y, y)
```

Wasted mem
transfers

Memory transfers

Manual memory transfer



```
from numba import cuda
```

```
@cuda.jit
```

```
def double_kernel(x, y):
```

```
    i = cuda.grid(1)
```

```
    y[i] = 2 * x[i]
```

```
x = np.random.rand(8 * 512)
```

```
→ d_x = cuda.to_device(x) # to GPU
```

```
→ d_y = cuda.device_array_like(d_x)
```

```
tpb = 512
```

```
bpg = len(x) // tpb
```

```
double_kernel[bpg, tpb](d_x, d_y)
```

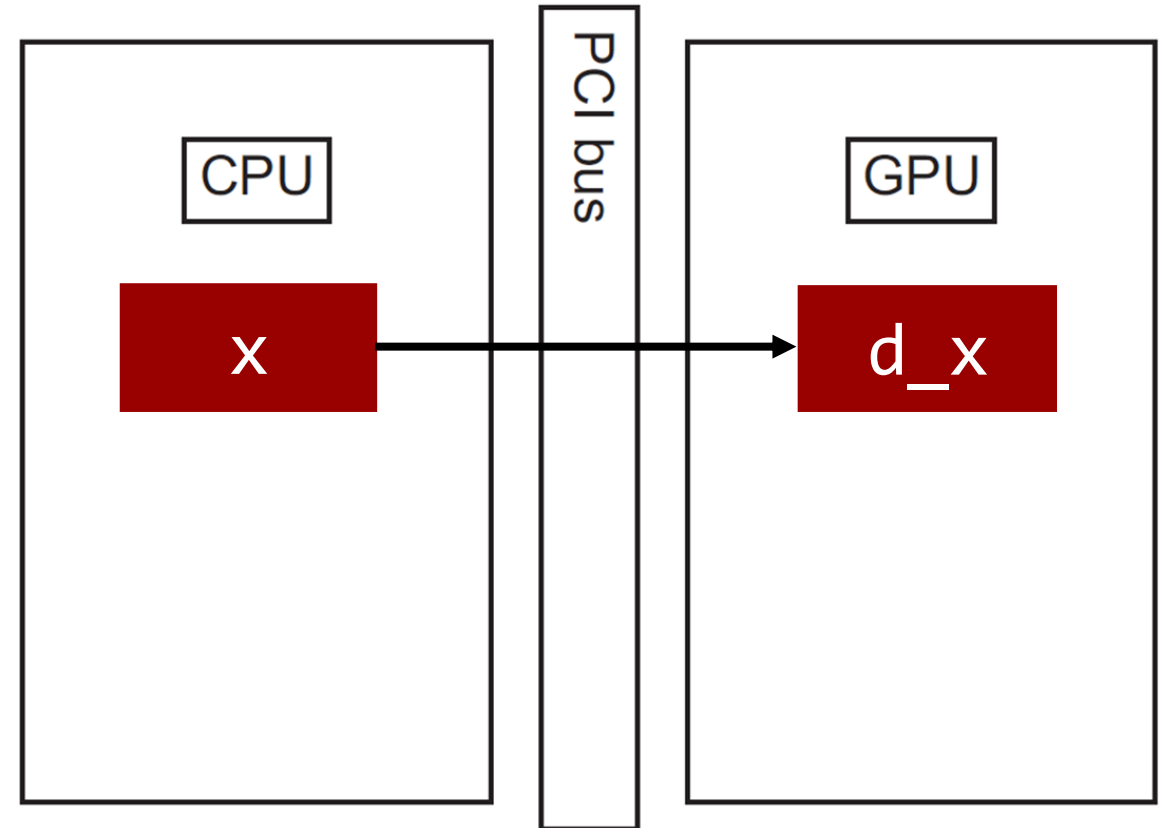
```
→ y = d_y.copy_to_host() # to CPU
```


Memory transfers

Pinned memory

Simplified (and wrong) view:
Data moves from CPU to GPU

Problem:
OS may swap out memory while
transfer is on-going



Memory transfers

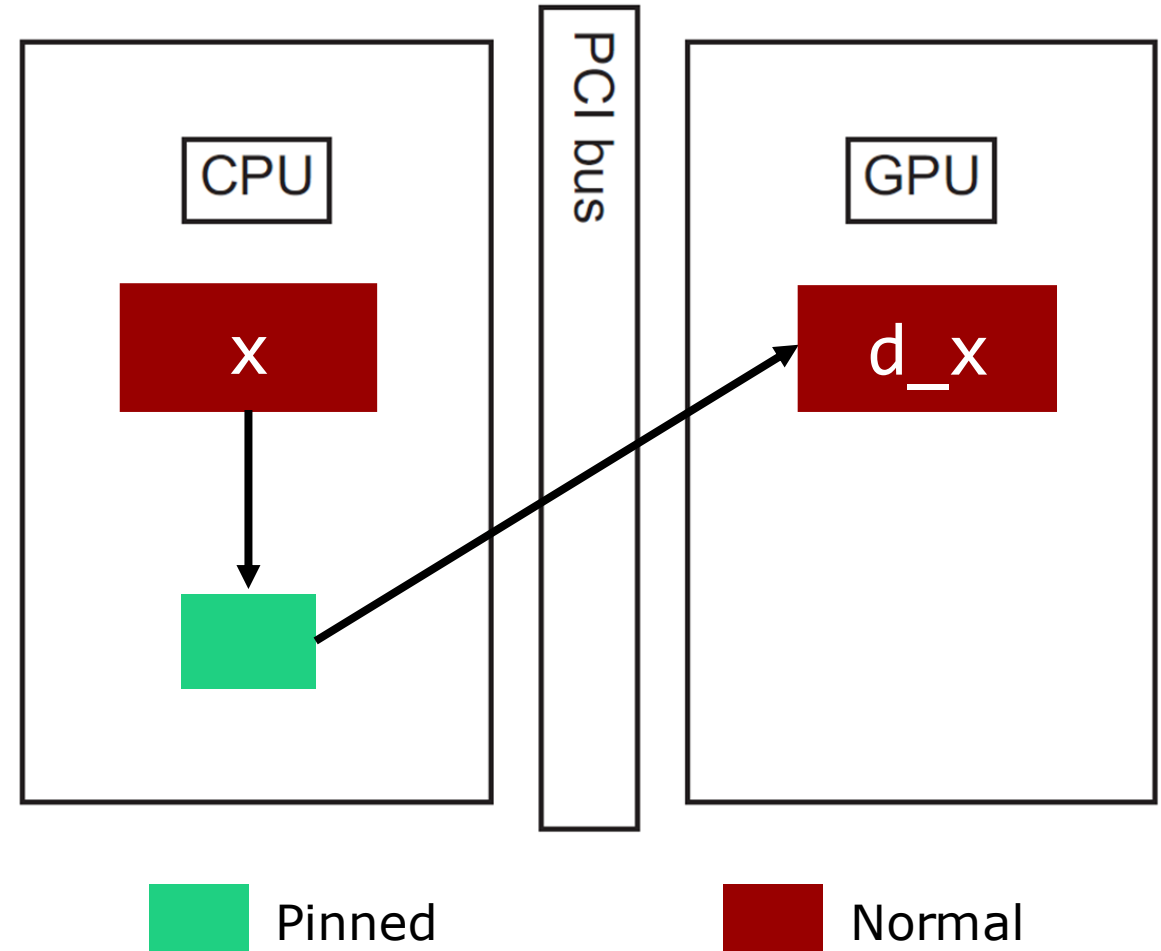
Pinned memory

Correct view:

Data moves from CPU to GPU
through pinned buffer – detour!

“Pinned” also called “page locked”

Ensures memory is kept in place
in physical RAM



Memory transfers

Pinned memory

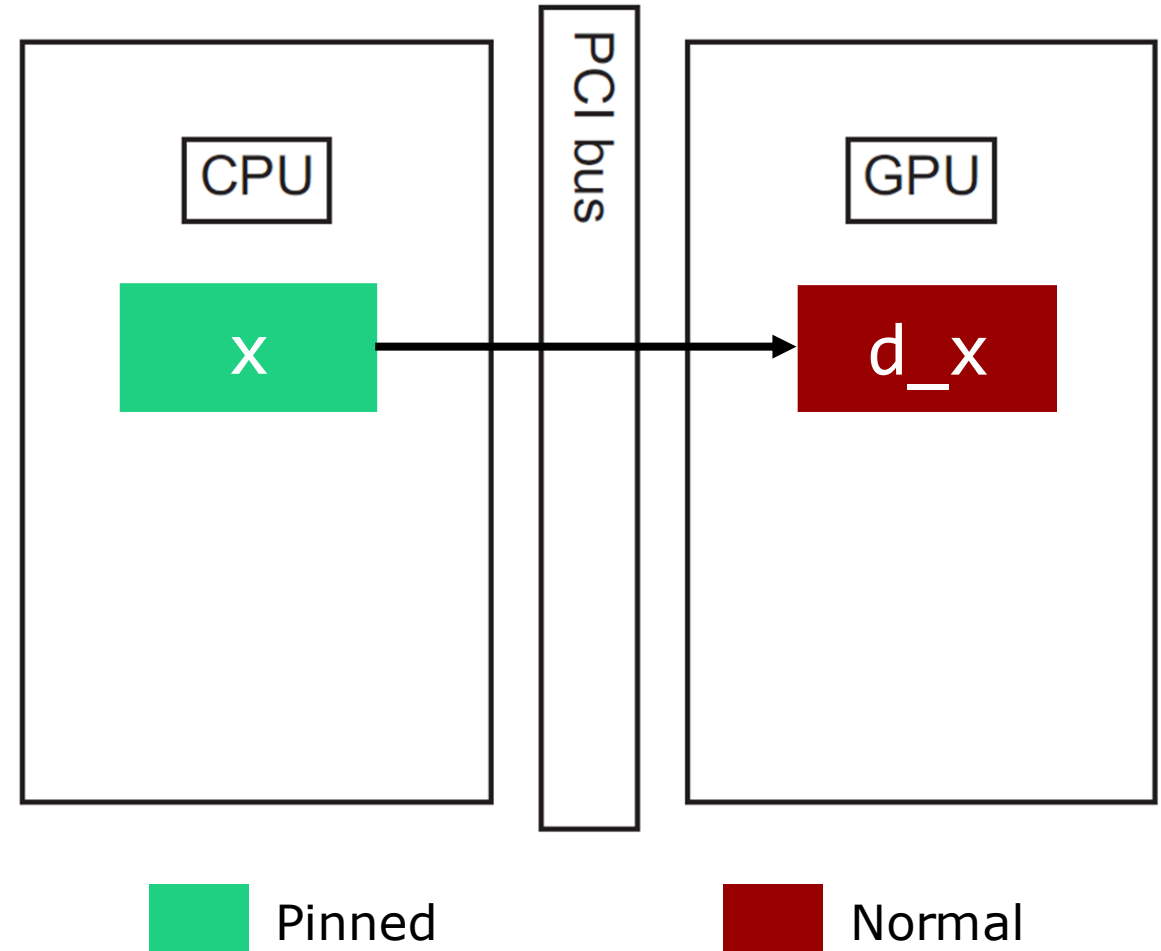
Solution:

Manually pin memory

```
from numba import cuda

# Pin existing
x = np.zeros(100)
with cuda.pinned(x):
    # x is now in pinned memory

# Allocate (uninitialized) pinned
xp = cuda.pinned_array(100)
```



Memory transfers

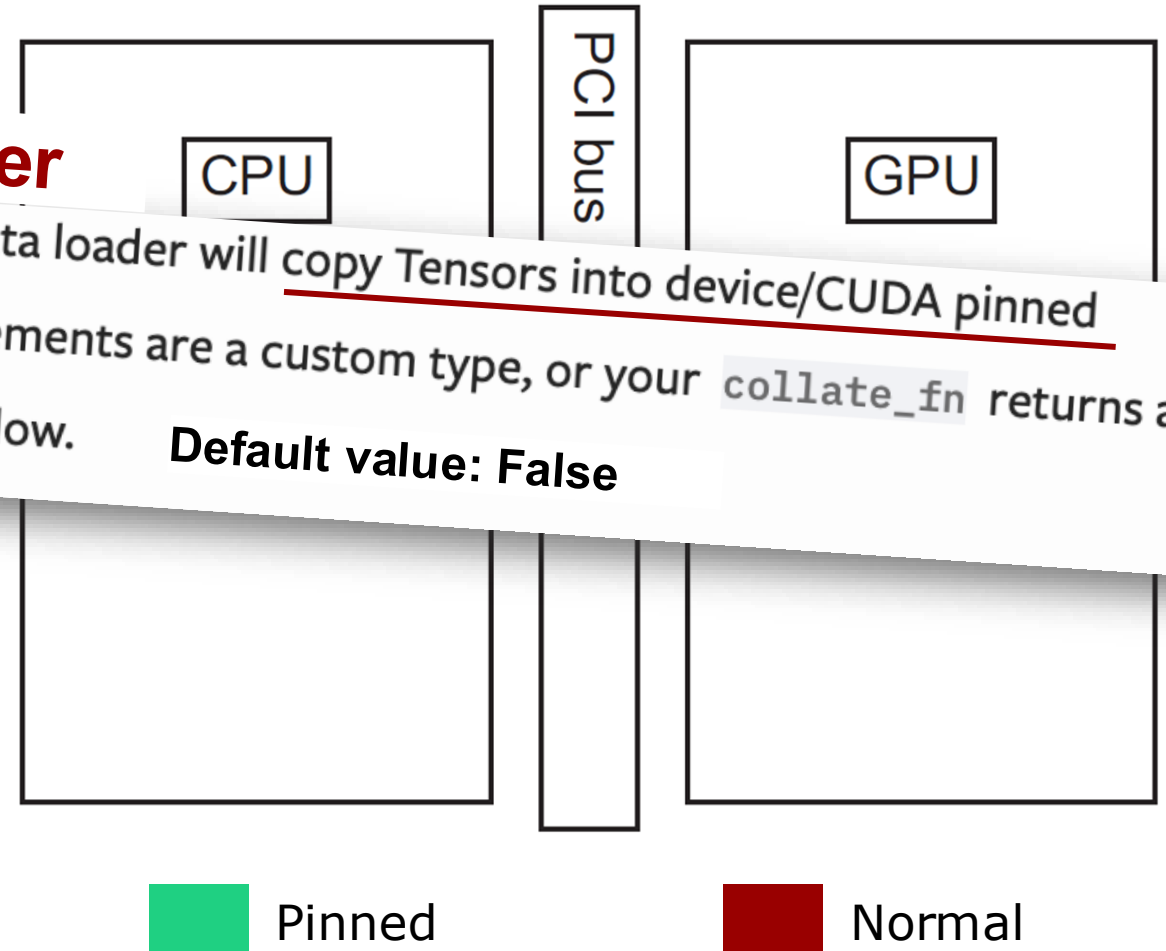
Pinned memory

Side-note: PyTorch DataLoader

- **pin_memory** (*bool, optional*) – If `True`, the data loader will copy Tensors into device/CUDA pinned memory before returning them. If your data elements are a custom type, or your `collate_fn` returns a batch that is a custom type, see the example below. **Default value: False**

```
# Pin existing
x = np.zeros(100)
with cuda.pinned(x):
    # x is now in pinned memory

# Allocate (uninitialized) pinned
xp = cuda.pinned_array(100)
```



Quiz time!

GPUs on DTU HPC

GPU job scripts

```
#!/bin/sh
#BSUB -q hpc
#BSUB -J job
#BSUB -n 1
#BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=1GB]"
#BSUB -W 00:15
#BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python program.py
```

GPU job scripts

Queue

4 cores

Request 1 GPU

Exclusive to your proces (job)

Max 30 minutes

```
#!/bin/sh
#BSUB -q c02613
#BSUB -J gpujob
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=1GB]"
#BSUB -gpu "num=1:mode=exclusive_process"
#BSUB -W 00:30
#BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err
```

```
source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613
```

```
python gpuprogram.py
```


GPU job scripts

Queue → #BSUB -q gpuv100 ← V100 GPUs
#BSUB -J gpujob 16GB GPU RAM
Min. 4 cores → #BSUB -n 4 32GB GPU RAM
Request 1 GPU
Exclusive to your proces (job) → #BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=1GB]"
#BSUB -gpu "num=1:mode=exclusive_process"
Max 24 hours → #BSUB -W 00:30
#BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python gpuprogram.py

GPU job scripts

Queue → #BSUB -q gpuv100 ← V100 GPUs
#BSUB -J gpujob 16GB GPU RAM
Min. 4 cores → #BSUB -n 4 32GB GPU RAM
#BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=1GB]"
Request 1 GPU → #BSUB -gpu "num=1:mode=exclusive_process"
Exclusive to your proces (job)
#BSUB -W 00:30
#BSUB -R "select[gpu32gb]" ← Select specific GPU
Max 24 hours
#BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python gpuprogram.py

GPU job scripts

Queue → #BSUB -q gpua100 ← A100 GPUs
#BSUB -J gpujob 40GB GPU RAM
Min. 4 cores → #BSUB -n 4 80GB GPU RAM
Request 1 GPU
Exclusive to your proces (job) → #BSUB -R "span[hosts=1]"
#BSUB -R "rusage[mem=1GB]"
#BSUB -gpu "num=1:mode=exclusive_process"
#BSUB -W 00:30
#BSUB -R "select[gpu80gb]" ← Select specific GPU
Max 24 hours → #BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err

```
source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613

python gpuprogram.py
```

GPU job scripts

```
#!/bin/sh
Queue → #BSUB -q gpua100 ← A100 GPUs
#BSUB -J gpujob 40GB GPU RAM
Min. 4 cores → #BSUB -n 4 80GB GPU RAM
#BSUB -R "span[hosts=1]"
Request 1 GPU → #BSUB -R "rusage[mem=1GB]"
Exclusive to your proces (job) → #BSUB -gpu "num=1:mode=exclusive_process"
#BSUB -W 00:30
Max 24 hours → #BSUB -R "select[gpu80gb]" ← Select specific GPU
#BSUB -o batch_output/gpujob_%J.out
#BSUB -e batch_output/gpujob_%J.err

source /dtu/projects/02613_2025/conda/conda_init.sh
conda activate 02613_2026 ← Note: updated environment for GPU!
                             More later...

python gpuprogram.py
```

GPU interactive nodes

```
$ sxm2sh
```

```
...
```

This is a shared machine.....

CUDA_VISIBLE_DEVICES is set to: 2

you can set CUDA_VISIBLE_DEVICES to something else, if you need it

for example: check first with 'nvidia-smi' and if you then decide to use GPU 1,3 do a

export CUDA_VISIBLE_DEVICES=1,3

(and don't put any CUDA_VISIBLE_DEVICES-lines in your jobscripts, they just don't belong there)

And as a general reminder: please don't leave anything running on these machine unless you debug something. Blocking huge amounts of GPU-memory is preventing other users to use/debug their GPU-codes. Same with jupyter-notebooks: Use them to do something, but then also please close them when finished.

The majority of our GPU-resources are in the batch-queues, so that's also how the GPUs should generally be used.

GPU interactive nodes

```
$ sxm2sh
```

```
...
```

```
This is a shared machine.....
```

```
CUDA_VISIBLE_DEVICES is set to: 2
```

```
... CUDA_VISIBLE_DEVICES to something else, if you need it
```

```
... decide to use GPU 1,3 do a
```

Note: only 3 interactive nodes for the entire uni!
PLEASE: Use these for testing, not for long jobs!

```
GPU-codes. Same as batch-queue  
when finished.
```

```
The majority of our GPU-resources are in the batch-queues, so that's also how the GPUS should  
generally be used.
```

GPU interactive nodes

```
$ sxm2sh
...
$ nvidia-smi
```

NVIDIA-SMI 570.124.06				Driver	.06	Util.	Version: 12.8
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Memory-Usage		GPU-Util	Compute M.	
		Pwr:Usage/Cap				MIG M.	
0	Tesla V100-SXM2-32GB	On	00000000:15:00.0	Off		0	
N/A	47C P0	81W / 300W	22042MiB / 32768MiB		45%	Default	N/A
1	Tesla V100-SXM2-32GB	On	00000000:16:00.0	Off		0	
N/A	44C P0	71W / 300W	10390MiB / 32768MiB		0%	Default	N/A
2	Tesla V100-SXM2-32GB	On	00000000:3A:00.0	Off		0	
N/A	32C P0	56W / 300W	21438MiB / 32768MiB		0%	Default	N/A

```
...
```

GPU interactive nodes

```
$ sxm2sh
...
$ export CUDA_VISIBLE_DEVICES=0      # Only use first GPU
$ export CUDA_VISIBLE_DEVICES=2      # Only use third GPU
$ export CUDA_VISIBLE_DEVICES=1,3    # Only use second and fourth GPU
```


GPU interactive nodes

```
$ sxm2sh # V100 32GB GPUs
...
$ export CUDA_VISIBLE_DEVICES=0 # Only use first GPU
$ export CUDA_VISIBLE_DEVICES=2 # Only use third GPU
$ export CUDA_VISIBLE_DEVICES=1,3 # Only use second and fourth GPU
```

```
$ voltash # V100 16GB GPUs
...
```

```
$ a100sh # A100 40GB GPUs
...
```

A100 in the c02613 queue

Summary

GPU entry point: kernel function

- Specifies what each thread does
- Mental mode: each thread computes one output element

Must specify thread grid when calling kernel

- Threads per block and blocks per grid
- Make sure to cover the data
- May need bounds check in kernel

For kernel performance:

- Remember computations run in warps
- Avoid warp divergence
- Ensure warps access data cache efficiently

For memory transfer performance:

- Minimize CPU $\leftrightarrow$ GPU transfers
- Keep memory on GPU
- Use pinned arrays

Python update

Driver update incompatible with 02613 environment

- CUDA in numba in the 02613 environment does not work with newest drivers
- Common issues:
 - Segmentation fault
 - Error messages from numba-code
- Updating only numba was not possible due to conflicting dependencies
- New environment created (more on next slides)
- Old environment remains in place, but **GPU will not work** here

New 02613_2026 environment

- To use the new environment, replace
 - `conda activate 02613`
 - `conda activate 02613_2026`
- Must be done for
 - All code using GPU
 - All code in mini-project
- Extensively tested, but if you have code that works in 02613 but not in 02613_2026
 - Send entire program as zip or tar to 02613@compute.dtu.dk
 - Ask me or TAs
 - Post on Piazza, but **do not** post mini-project code!
- Also, `conda_init.sh` now informs you if you are on a node without a GPU

What's new?

- Python 3.12 instead of 3.11, most packages have only minor version changes
- Pandas updated to version 3.x from 2.x
 - Text-columns now have dtype str instead of object
 - Uses less memory – converting to, e.g., datetime still beneficial
 - Converting to datetime does not always result in nanosecond resolution
 - Resolution inferred from data – seconds, milli-, micro-, nanoseconds, ...
 - Be careful converting to integers!
 - Check the `as_unit` function if needed.
 - Several other changes, but none impacting course exercises
 - Change log: <https://pandas.pydata.org/docs/whatsnew/v3.0.0.html>

Why update?

- Preferable to not update during course/project/...
- Exception do apply: New useful features, performance improvements, bug-fixes, ...
- Unexpected effects may occur: Need to test everything again
- This update forced by driver incompatibility – numba.cuda simply stopped working
- HPC system has 1000s of users – kept up-to-date for security
- This can happen anywhere – hobby-project, university, industry, ...

What should you take away from this?

- The world is not static: Your environment may (will) break due to external changes
- You may need to update your environment and your code...
- ...even if you did not initiate or request the external change
- Being able to adapt can even increase your market value as an engineer

Today's exercise

Numba JIT and GPU Computing

- Warmup with CPU JIT
- GPU basics with 1D vector addition
- Thread grids with matrix-matrix multiplication

Please work in groups for GPU exercises!

Have our *own* queue for GPU:
-q c02613

Max wall time: 30 minutes

Min cores: -n 4

GPU mem.: 5GB

Public queues (very busy):
-q gpuv100 and -q gpua100

Interactive GPUs:

Public: voltash, sxm2sh, a100sh

Numba JIT and GPU Computing

Note on timing JIT compiled code

```
func(x)  # Ensure func is compiled

t = time()
for _ in range(100):
    func(x)
t = time() - t
```

Numba JIT and GPU Computing

Note on timing GPU JIT compiled code

```
func(x)  # Ensure func is compiled

t = time()
for _ in range(100):
    func(x)
cuda.synchronize()  # Ensure all kernels are done
t = time() - t
```

Numba JIT and GPU Computing

Numba CUDA simulator

```
# Before importing numba
import os
os.environ['NUMBA_ENABLE_CUDASIM'] = '1'

import numba
```

Useful concepts

CPU JIT

```
@jit(nopython=True)
def func(x):
    ...
```

CUDA JIT

```
@cuda.jit
def func(x, y):
    i = cuda.grid(1)
    ...
    y[i] = ... # Write to out array
```

```
func[blockspergrid, threadsperblock](x, y)
```

New Python environment

```
conda activate 02613_2026
```

Change to GPU work node

```
voltash
sxm2sh
a100sh <- same GPU as in course GPU queue
```

GPU queue

```
c02613
```

Must add bsub option:

```
#BSUB -gpu "num=1:mode=exclusive_process"
```

Submit job script

```
bsub < submit.sh
```

Max wall time: 30 minutes

Max nr. of cores: 4